CS1312
Algorithms Design and Analysis

Greedy Graph Algorithms

Types of Problems

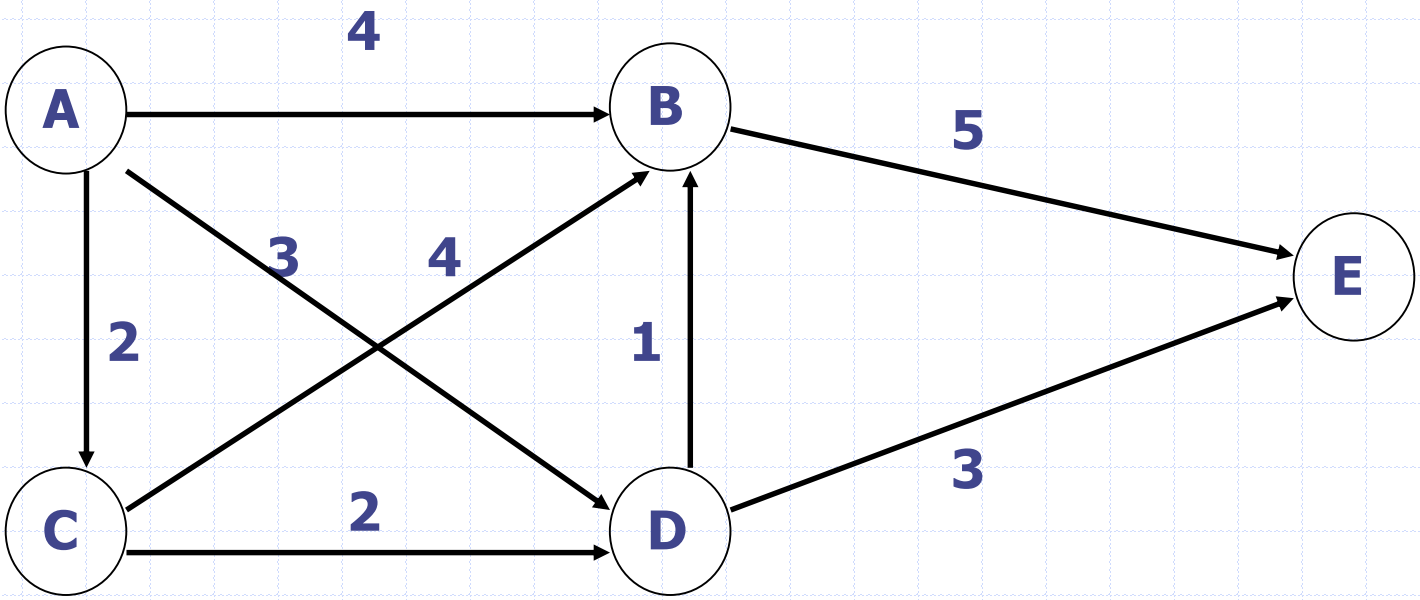
Problems we are trying to solve are basically of two kinds:

- **Decision problems:** trying to decide whether a statement is true or false.
  - They have either **yes** or **no** as the solution.

- **Optimization problems:** trying to find the solution with the **best** possible score according to some scoring scheme.
  - Maximization problems: trying to maximize a certain score.
  - Minimization problems: trying to minimize a cost function.

Types of Problems - example

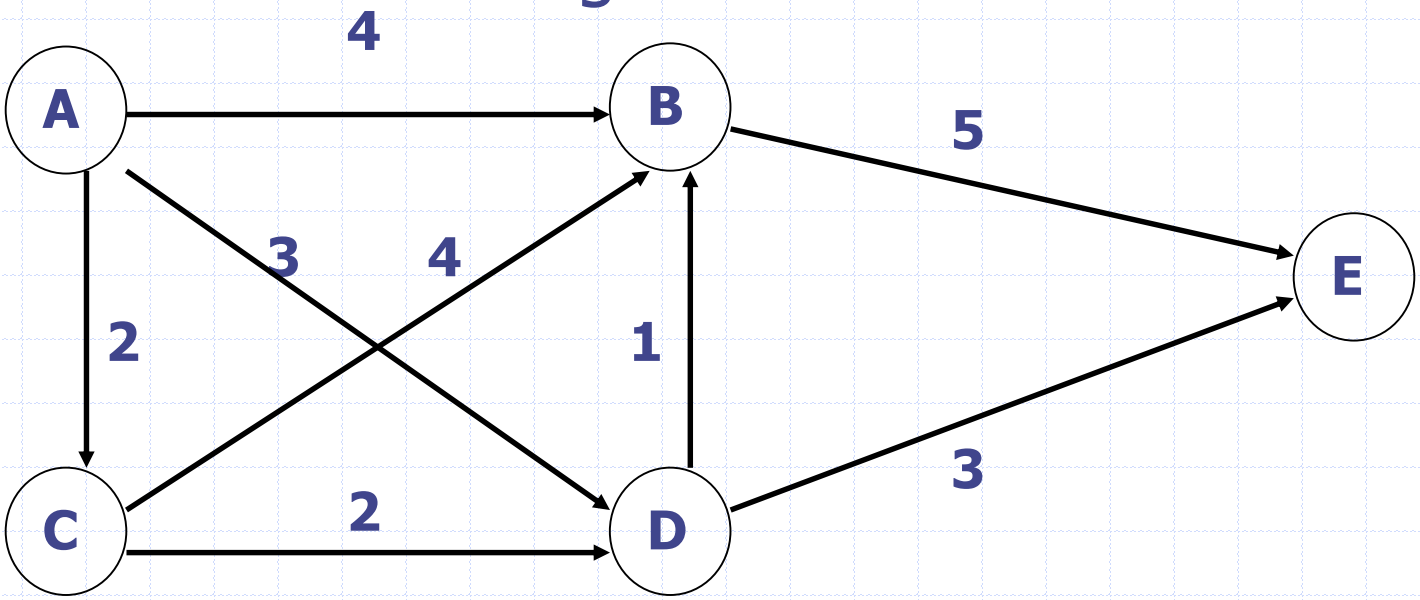
- Example: Find the shortest path from A to E



Decision Problem?
Optimization Problem?

Types of Problems - example

- Example: does a path exist in G between A and E whose length is at most k?



Decision Problem?
Optimization Problem?

Optimization Problems

- Optimization: choosing the best element from a set of alternatives.

- An **optimization algorithm** is a numerical method or algorithm for finding a value x such that f(x) is as small (or as large) as possible, for a given function f, possibly with some constraints on x.

Optimization Problems

- For an optimization problem you want to find, not just *a* solution, but the *best* solution.

- For solving Optimization problems:
  1. Greedy algorithm
  2. Dynamic programming algorithm
  3. Branch and bound algorithm (BFS)

Greedy Algorithms

- Algorithms for optimization problems typically go through a sequence of steps, with a set of choices at each step.
- A greedy algorithm is used to solve optimization problems by always making the choice that looks best at the moment.
  - It makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution.
  - Example: buy a car or test a person for a career

Greedy Algorithms

- Greedy algorithms do not always yield optimal solutions, but for many problems they do.
- The greedy method is quite powerful and works well for a wide range of problems.

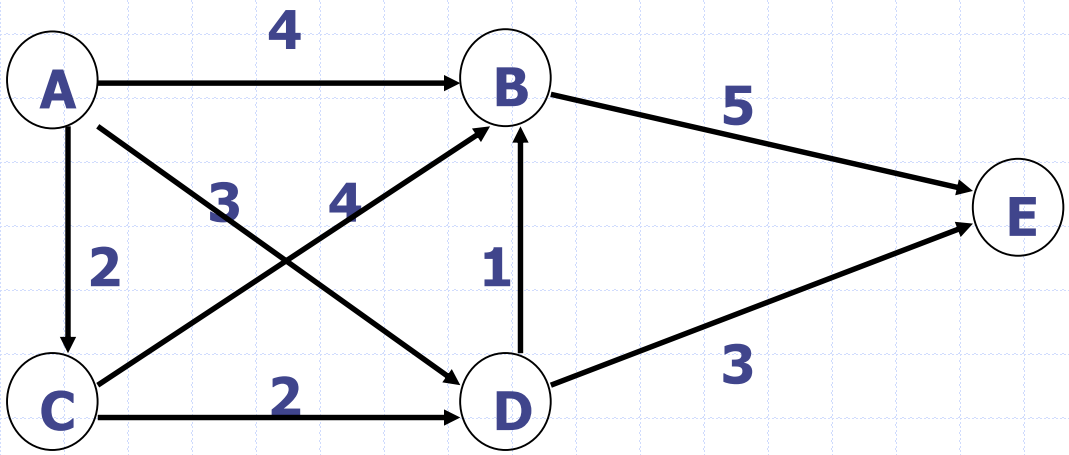
Greedy Algorithms

- Greedy Algorithms are used to solve optimization problems like smallest, longest, largest … etc.

- Examples of Greedy Algorithms :
  - Minimum-spanning-tree algorithms
  - Shortest path problems.
  - Travelling salesman problem.
  - The knapsack problem (items are divisible).
  - Task scheduling problem.
  - Huffman Coding.

Greedy Algorithms

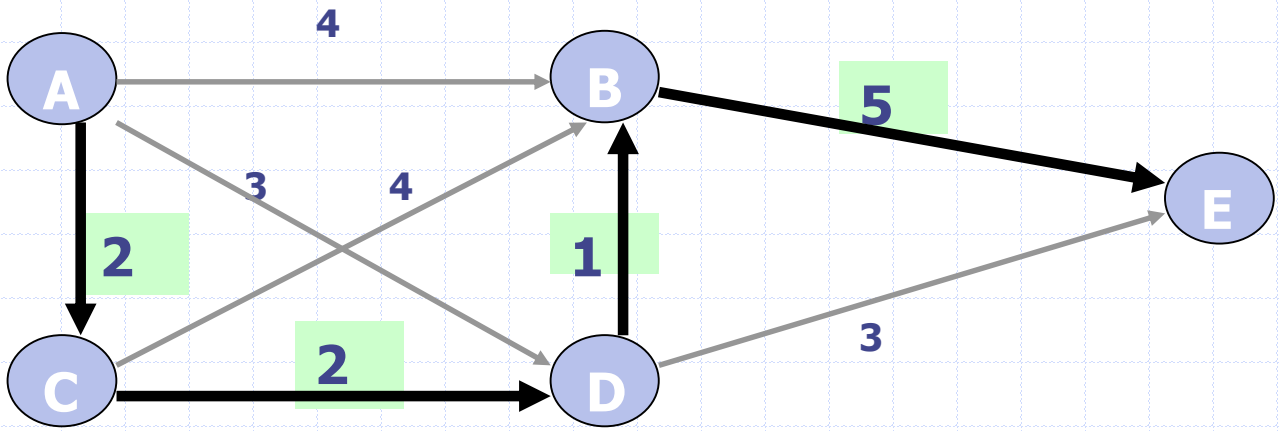
Example: Find the shortest path from A to E using A greedy method.



Greedy Algorithms - example

Greedy solution :

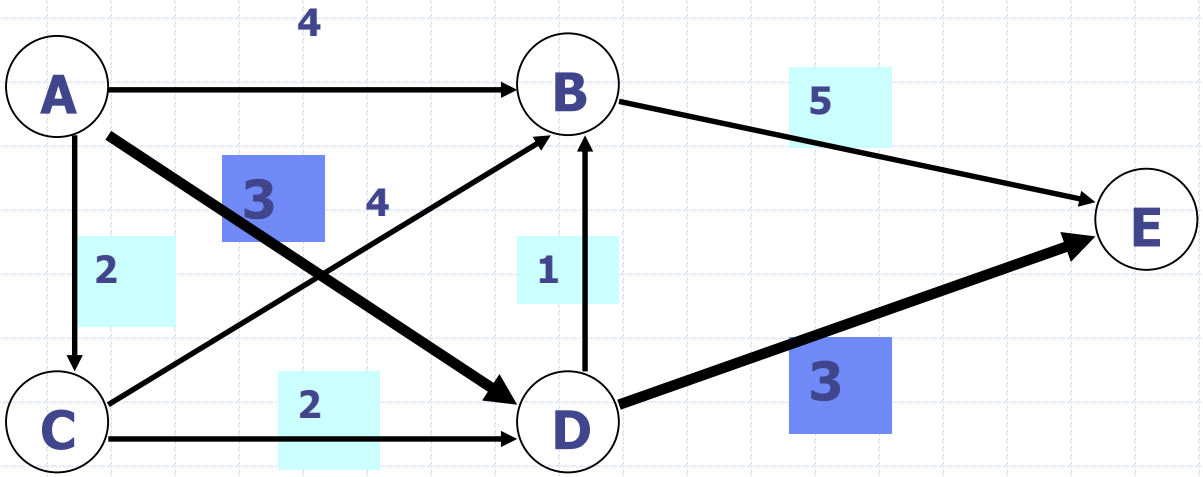
- Order of visit is A → C → D → B → E
- Path cost is 2+2+1+5 = 10



Greedy Algorithms - example

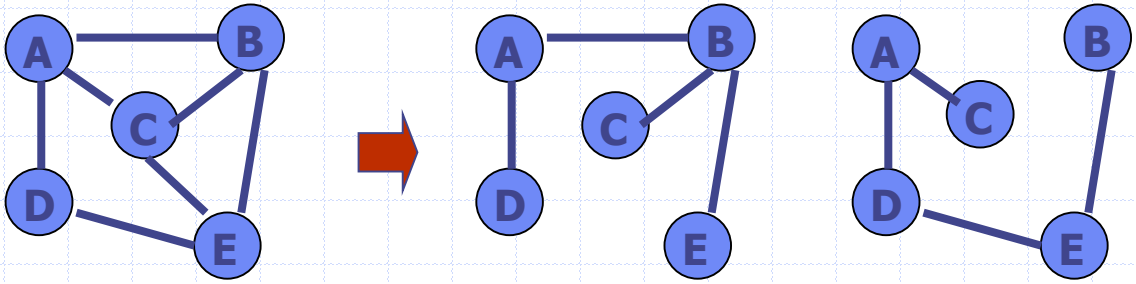
In the example the optimal solution is

- A → D → E
- Path cost is 3+3 = 6



Spanning Trees

- A Tree is a connected undirected graph that contains no cycles.
- A Spanning Tree of a graph G is a subgraph of G that is a tree and contains all the vertices of G

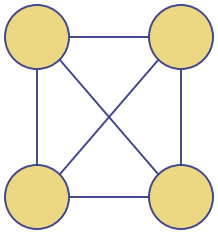


Undirected Graph **Some Spanning Trees**

Properties:

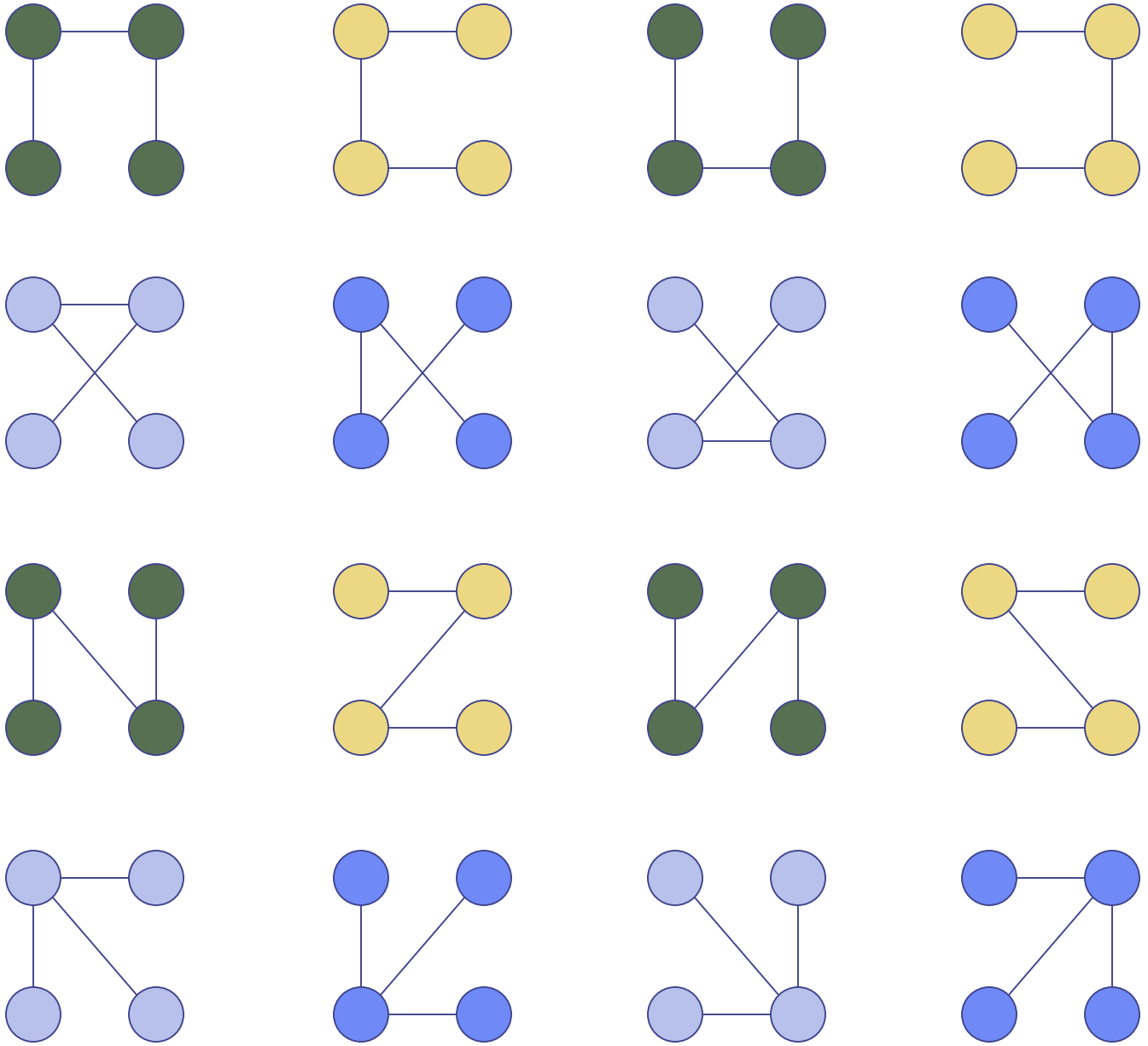
- The spanning tree of a n –vertex Undirected Graph has exactly n-1 edges
- It connects all the Vertices in the Graph
- A Spanning tree has no Cycles

Graph



Note: G has many MST when G contains cycles.

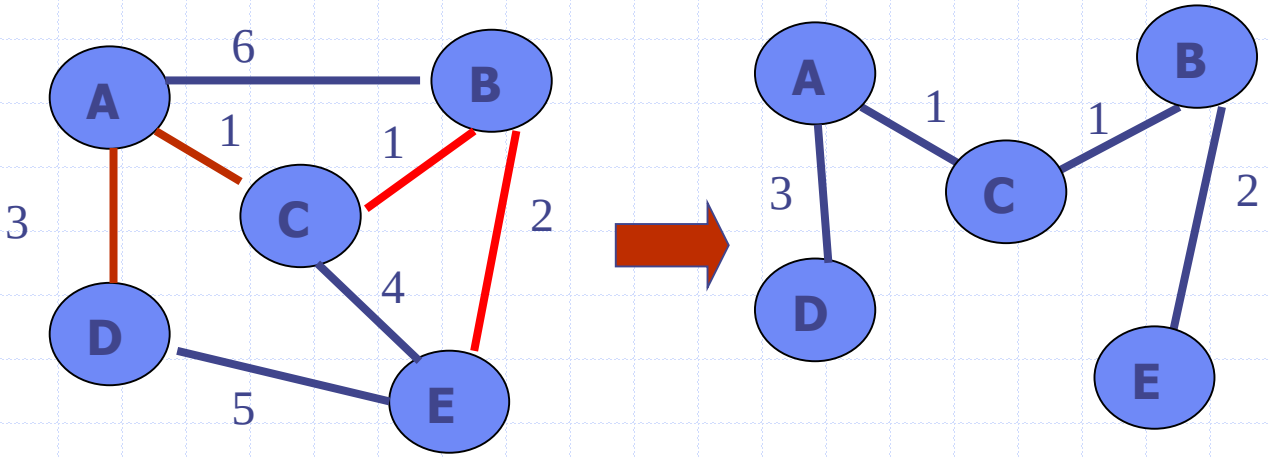
All 16 of its Spanning Trees



Minimum Spanning Trees

Minimum Cost Spanning Tree (MCST):

Is the tree among all the spanning trees with the *Lowest Cost.*



Weighted Undirected Graph **MCST**

Minimum Spanning Trees, why?

Example: Electronic circuit designs often need to make the pins of several components electrically equivalent by wiring them together. To interconnect a set of n pins, we can use an arrangement of n-1 wires, each connecting two pins. Of all such arrangements, the one that uses the least amount of wire is usually the *most desirable.*

Minimum Spanning Trees, why?

This wiring problem could be modeled with a connected, undirected graph G(V,E) where V is the set of pins, E is the set of possible interconnections between pairs of pins, and for each edge (u,v), we have a weight w specifying the cost (amount of wire needed) to connect u and v. We then wish to find an acyclic subset T € E that connects all of the vertices and *whose total weight* is *minimized.*

$$w(T) = \sum_{(u,v)\in T} w(u,v)$$

Minimum Spanning Trees, why?

- Computer Networks
  - To find how to connect a set of computers using the minimum amount of wire.
- Shipping/Airplane Lines
  - To find the fastest way between locations.

Building Minimum Spanning Trees

- Assume that we have a connected, undirected graph G (V,E) with a weight function w: E → **R** , and we wish to find a minimum spanning tree for G.
- We will study two algorithms that use a *greedy approach* to the problem:
  - Kruskal's algorithm (Joseph Kruskal in 1956)
  - Prim's algorithm (1930 by Czech mathematician Vojtěch Jarník and later rediscovered and republished by computer scientists Robert C. Prim in 1957 and Edsger W. Dijkstra in 1959)
- This greedy strategy is captured by the following generic method, which grows the minimum spanning tree one edge at a time.

Building Minimum Spanning Trees

- At each step, we determine an edge (u, v) (safe edge) that we can add to A without violating the spanning tree, in the sense that A ∪ $\{(u,v)\}$ also a subset of a minimum spanning .

```
GENERIC-MST(G, w)
1  A = ∅
2  while A does not form a spanning tree
3      find an edge (u, v) that is safe for A
4      A = A ∪ {(u, v)}
5  return A
```

Kruskal's Algorithm

- This algorithm creates a forest that consists of n single node (and no edges).
- The **safe edge** added to A is always a least-weight (cheapest) edge in the graph that connects two distinct components (trees).
- If it were to form a cycle, it would simply link two nodes that were already part of a single connected tree, so that this edge would not be needed.
- Kruskal's algorithm qualifies as a **greedy algorithm** because at each step it adds to the forest an edge of **least possible weight**.

Kruskal's Algorithm

Steps of the algorithm:

- Initialize the set A = {} to the empty set.
- Create |V| trees, one containing each vertex.
- Sort all edges in non-decreasing order (from lowest to highest).
- For each edge (u,v) examines whether the endpoints u and v belong to the same tree:
  - If they at the same tree , then the edge (u,v) cannot be added to the forest without creating a cycle, and the edge is discarded.
  - Otherwise, the two vertices belong to different trees, and the edge is added to the tree.

Kruskal's Algorithm

```
MST-KRUSKAL(G, w)
1  A = ∅
2  for each vertex v ∈ G.V
3       MAKE-SET(v)
4  sort the edges of G.E into nondecreasing order by weight w
5  for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
6       if FIND-SET(u) ≠ FIND-SET(v)
7            A = A ∪ {(u, v)}
8            UNION(u, v)
9  return A
```

Algorithm Make-Set(x)
{ creates a new set whose only member is x;
}
Algorithm Union(x, y)
{ unites the set containing x with the set containing y
}
Algorithm Find-Set(x)
{ returns a pointer to the set containing x
}

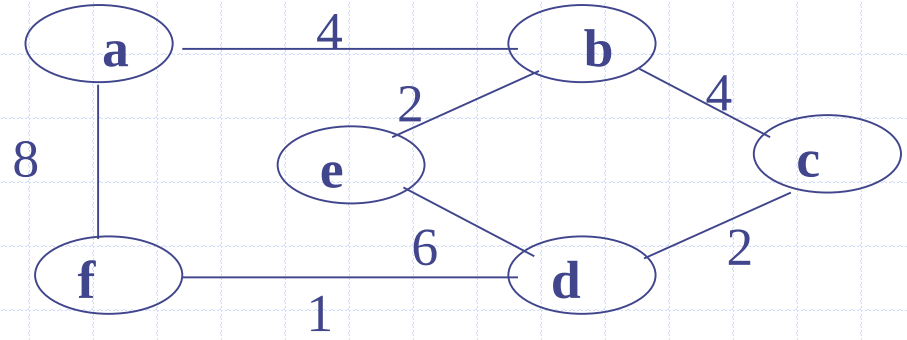
Kruskal's Algorithm: example 1

Initially

A = Φ, **Sets –** {a}, {b}, {c}, {d}, {e}, {f}.

E (Sorted in Ascending Order)

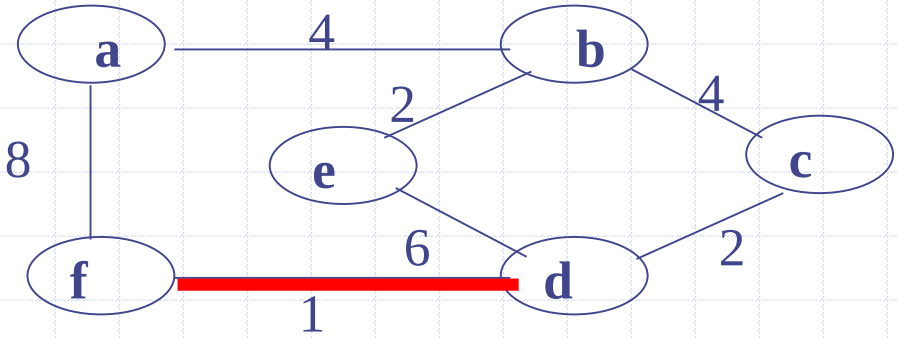
(f,d)	(b,e)	(c,d)	(a,b)	(b,c)	(e,d)	(a,f)



Kruskal's Algorithm: example 1

Step 1 **Sets –** {a}, {b}, {c}, {d}, {e}, {f}.

(f,d)	(b,e)	(c,d)	(a,b)	(b,c)	(e,d)	(a,f)



Take (f, d); Set(f) ≠ Set(d)
=> **Add** (f, d) to A,
Combine Set(f) & Set(d);

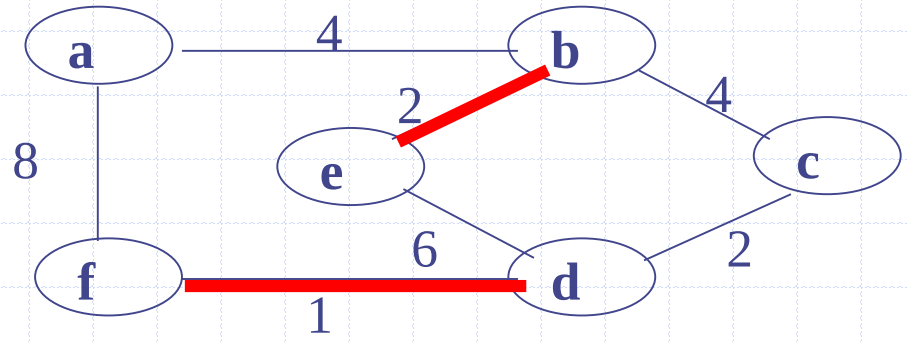
A = {(f, d)}.

Sets – {a}, {b}, {c}, {e}, {f, d}.

Kruskal's Algorithm: example 1

Step 2: **Sets –** {a}, {b}, {c}, {e}, {f, d}.

(f,d)	(b,e)	(c,d)	(a,b)	(b,c)	(e,d)	(a,f)



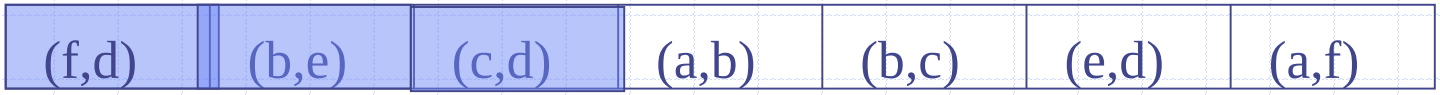
Take (b, e); Set(b) ≠ Set(e)
=> **Add** (b, e) to A,
Combine Set(b) & Set(e);

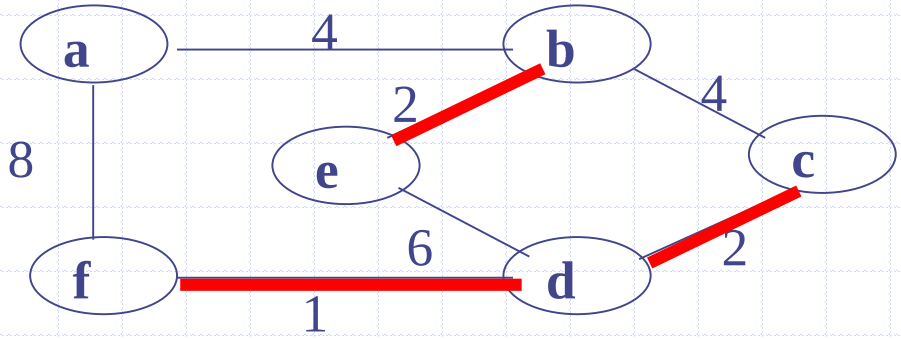
A = {(f, d), (b, e)}.

Sets – {a}, {b, e}, {c}, {f, d}.

Kruskal's Algorithm: example 1

Step 3: Sets – {a}, {b, e}, {c}, {f, d}.





Take (c, d); Set(c) ≠ Set(d)

=> **Add** (c, d) to A,

Combine Set(c) & Set(d);

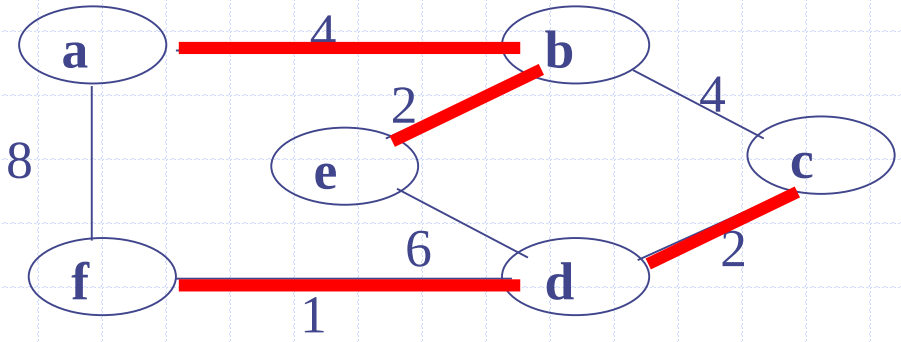
A = {(f, d), (b, e), (c, d)}.

Sets – {a}, {b, e}, {f, d, c}.

Kruskal's Algorithm: example 1

Step 4: Sets – {a}, {b, e}, {f, d, c}.

(f,d)	(b,e)	(c,d)	(a,b)	(b,c)	(e,d)	(a,f)



Take (a, b); Set(a) ≠ Set(b)
=> **Add** (a, b) to A
Combine Set(a) & Set(b);

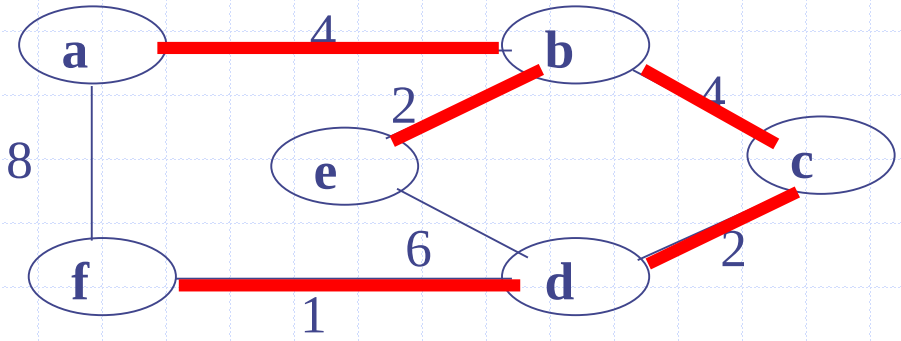
A = {(f, d), (b, e), (c, d), (a, b)}.

Sets – {b, e, a}, {f, d, c}.

Kruskal's Algorithm: example 1

Step 5: Sets – {b, e, a}, {f, d, c}.

(f,d)	(b,e)	(c,d)	(a,b)	(b,c)	(e,d)	(a,f)



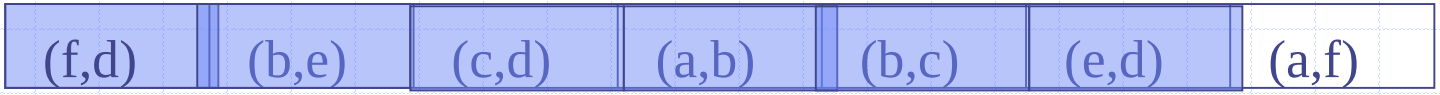
Take (b, c); Set(b) ≠ Set(c)
=> **Add** (b, c) to A,
Combine Set(b) & Set(c);

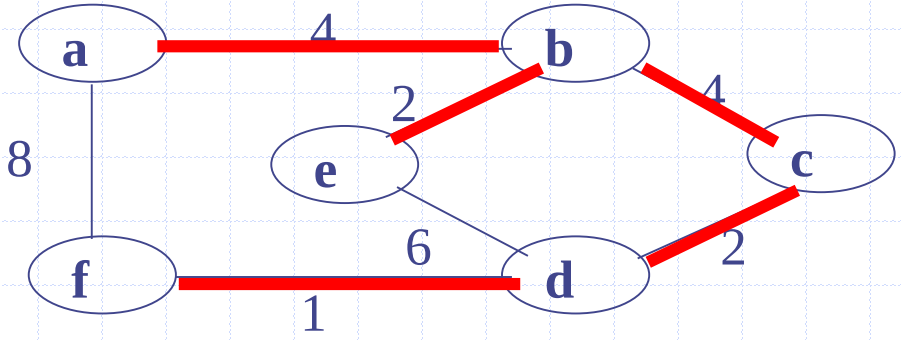
A = {(f, d), (b, e), (c, d), (a, b), (b, c)}.

Sets – {b, e, a, f, d, c}.

Kruskal's Algorithm: example 1

Step 6: **Sets –** {b, e, a, f, d, c}.





Take (e, d); Set(e) = Set(d)
=> **Ignore**

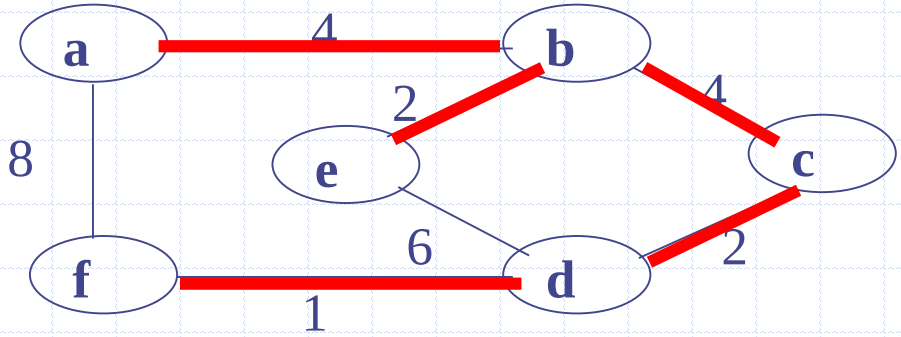
A = {(f, d), (b, e), (c, d), (a, b), (b, c)}.

Sets – {b, e, a, f, d, c}.

Kruskal's Algorithm: example 1

Step 7:

(f,d)	(b,e)	(c,d)	(a,b)	(b,c)	(e,d)	(a,f)



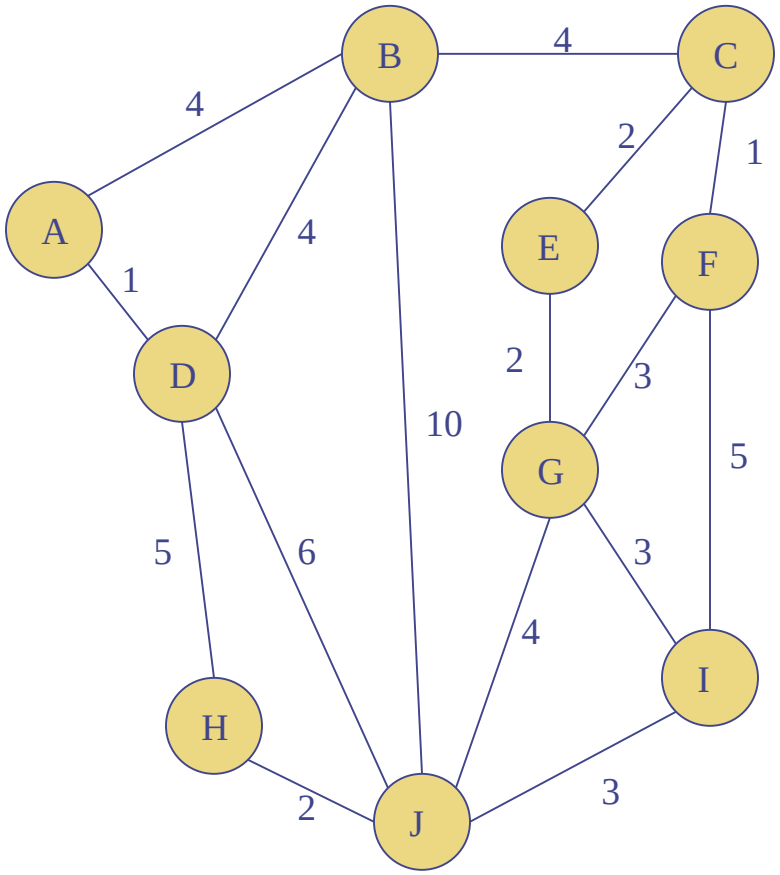
Take (a, f); Set(a) = Set(f)
=> **Ignore**

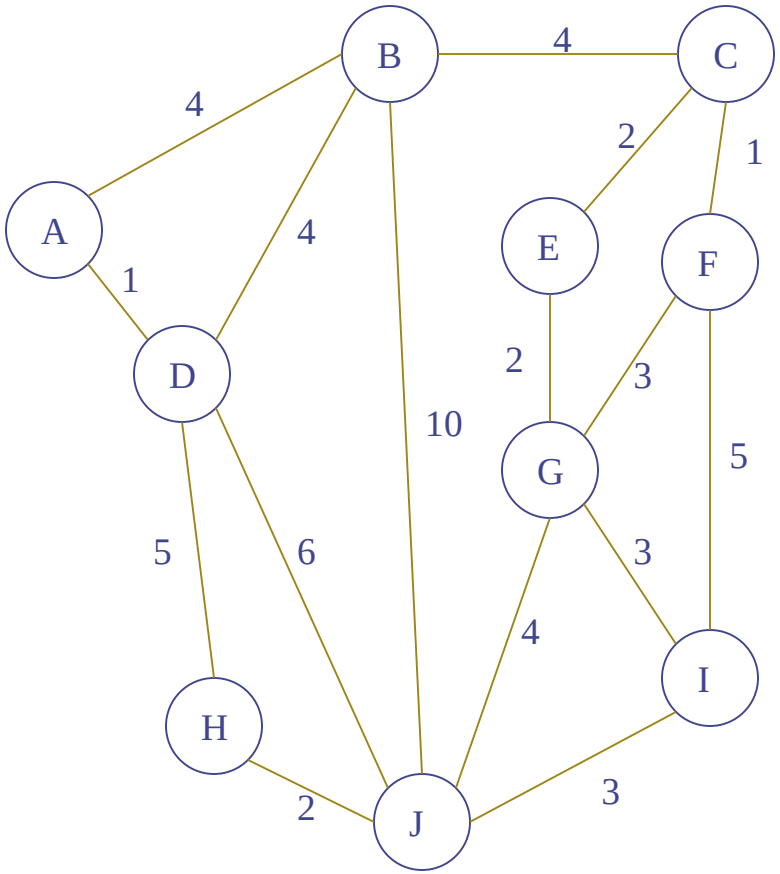
A = {(f, d), (b, e), (c, d), (a, b), (b, c)}.

Sets – {b, e, a, f, d, c}.

Value of MCST = 4 + 4 + 2 + 2 + 1 = 13.

Kruskal's Algorithm: example 2





A — 4 — B

A — 1 — D

B — 4 — C

B — 4 — D

B — 10 — J

C — 2 — E

C — 1 — F

D — 5 — H

D — 6 — J

E — 2 — G

F — 3 — G

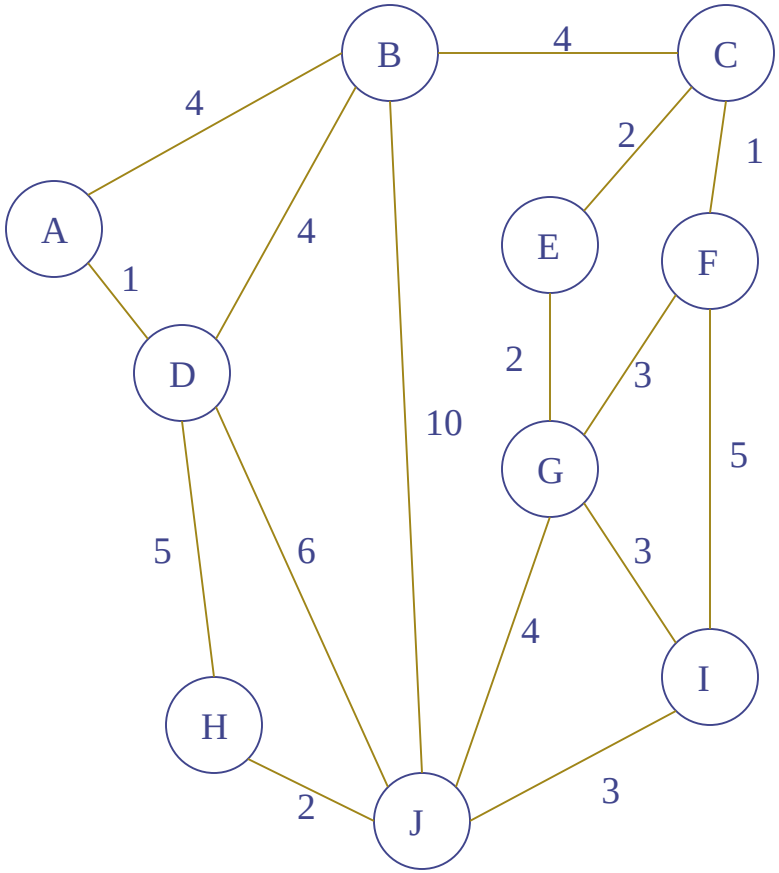
F — 5 — I

G — 3 — I

G — 4 — J

H — 2 — J

I — 3 — J



A — 1 — D

C — 1 — F

C — 2 — E

E — 2 — G

H — 2 — J

F — 3 — G

G — 3 — I

I — 3 — J

A — 4 — B

B — 4 — D

B — 4 — C

G — 4 — J

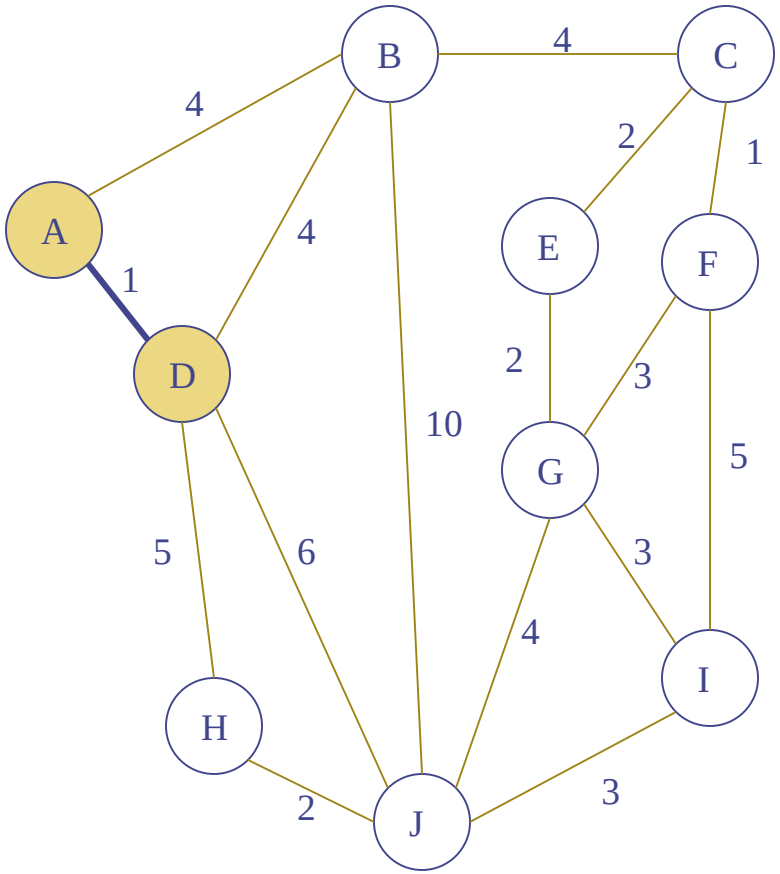
F — 5 — I

D — 5 — H

D — 6 — J

B — 10 — J

Add Edge



A — 1 — D

C — 1 — F

C — 2 — E

E — 2 — G

H — 2 — J

F — 3 — G

G — 3 — I

I — 3 — J

A — 4 — B

B — 4 — D

B — 4 — C

G — 4 — J

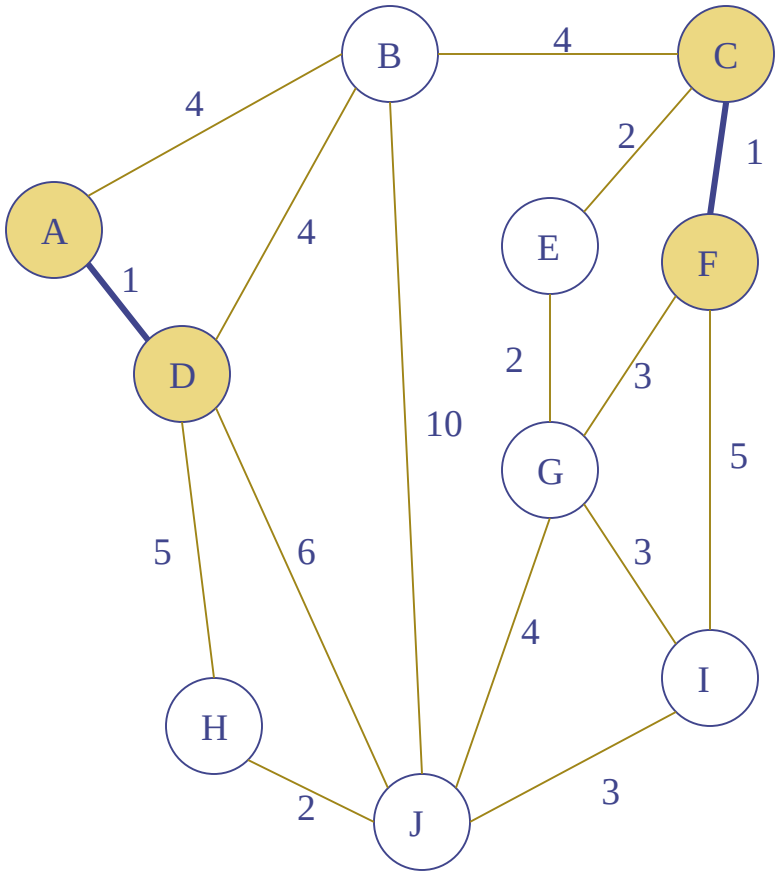
F — 5 — I

D — 5 — H

D — 6 — J

B — 10 — J

Add Edge



A — 1 — D

C — 1 — F

C — 2 — E

E — 2 — G

H — 2 — J

F — 3 — G

G — 3 — I

I — 3 — J

A — 4 — B

B — 4 — D

B — 4 — C

G — 4 — J

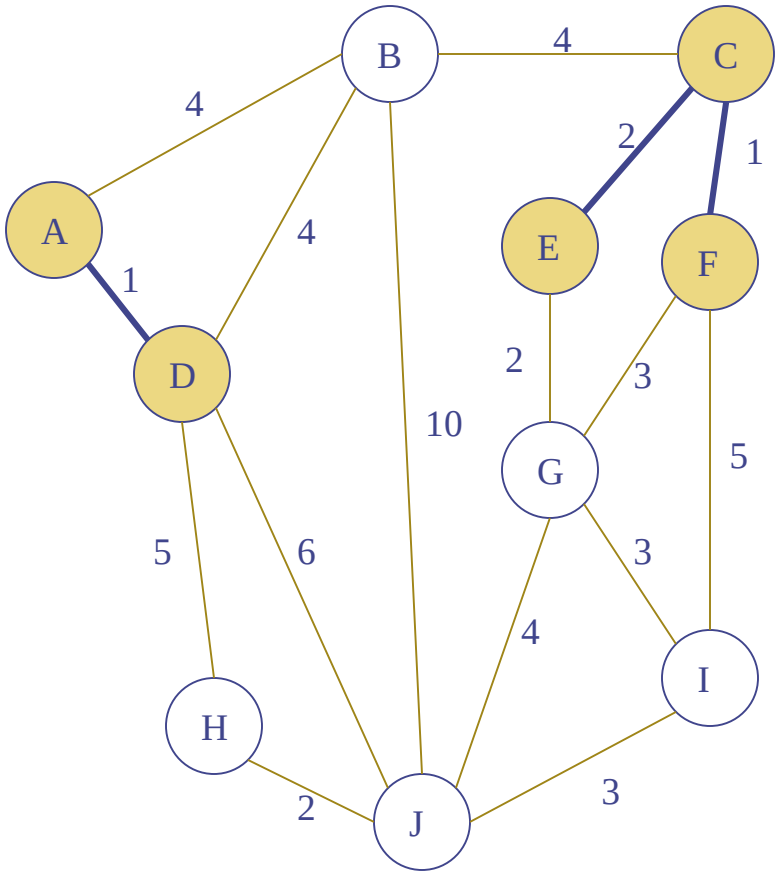
F — 5 — I

D — 5 — H

D — 6 — J

B — 10 — J

Add Edge



A — 1 — D

C — 1 — F

C — 2 — E

E — 2 — G

H — 2 — J

F — 3 — G

G — 3 — I

I — 3 — J

A — 4 — B

B — 4 — D

B — 4 — C

G — 4 — J

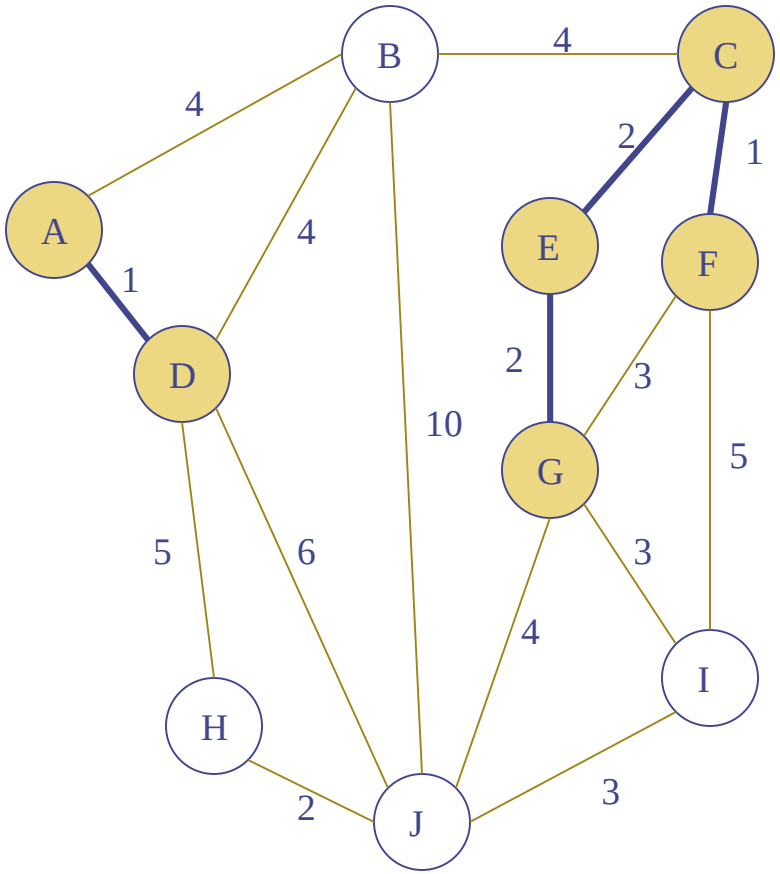
F — 5 — I

D — 5 — H

D — 6 — J

B — 10 — J

Add Edge



A — 1 — D

C — 1 — F

C — 2 — E

E — 2 — G

H — 2 — J

F — 3 — G

G — 3 — I

I — 3 — J

A — 4 — B

B — 4 — D

B — 4 — C

G — 4 — J

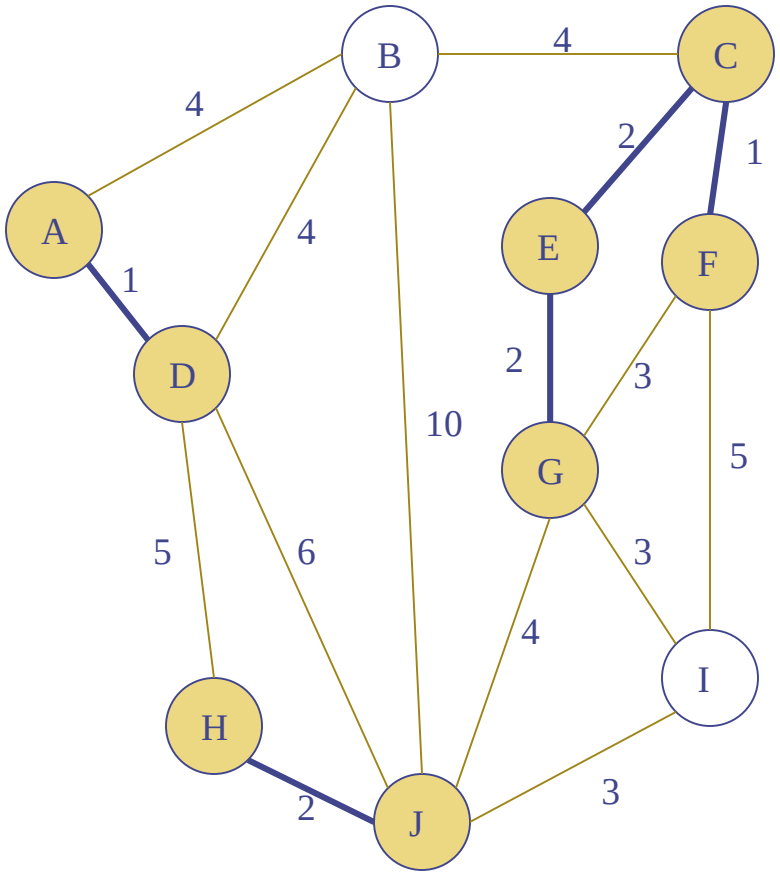
F — 5 — I

D — 5 — H

D — 6 — J

B — 10 — J

Add Edge



A — 1 — D

C — 1 — F

C — 2 — E

E — 2 — G

H — 2 — J

F — 3 — G

G — 3 — I

I — 3 — J

A — 4 — B

B — 4 — D

B — 4 — C

G — 4 — J

F — 5 — I

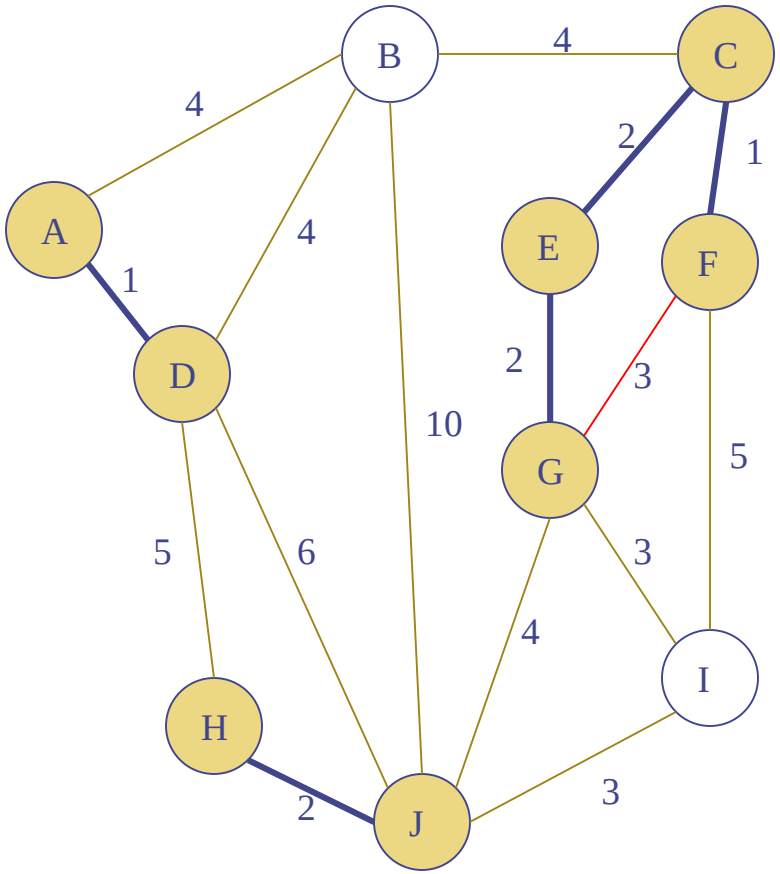
D — 5 — H

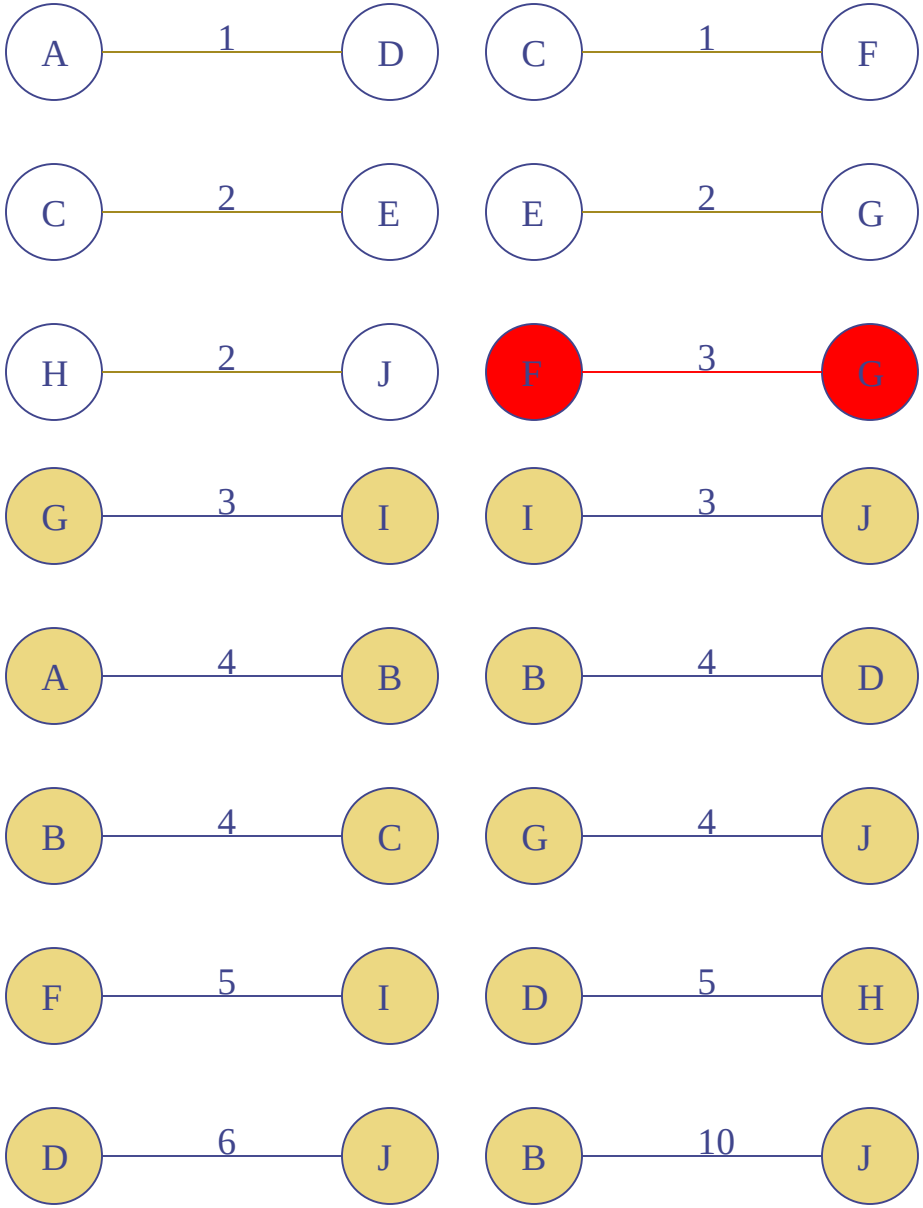
D — 6 — J

B — 10 — J

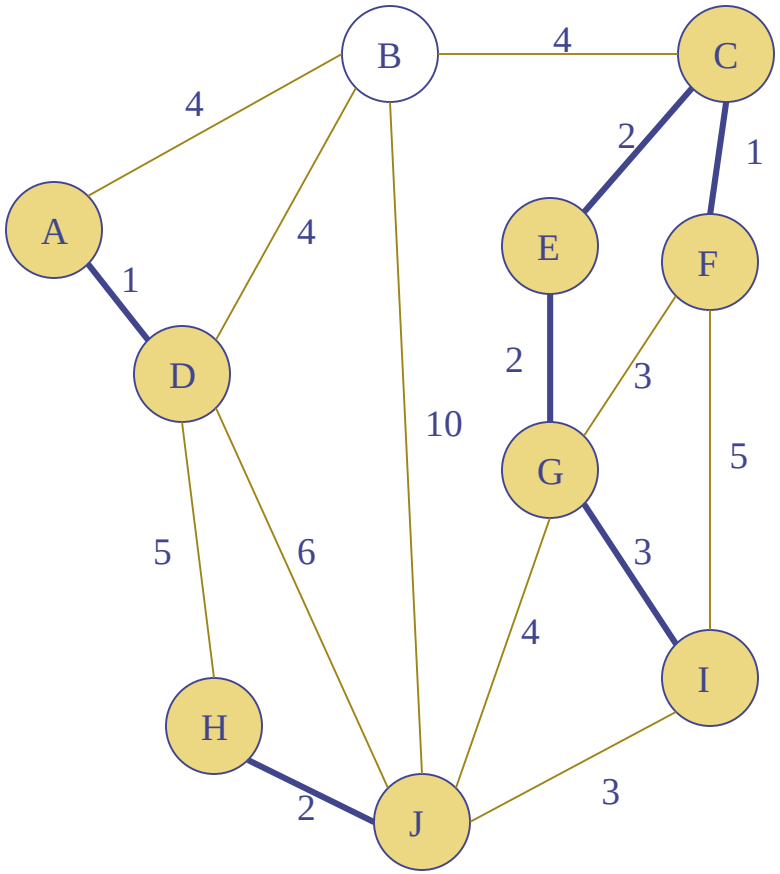
Cycle

Don’t Add Edge





Add Edge



A — 1 — D

C — 1 — F

C — 2 — E

E — 2 — G

H — 2 — J

F — 3 — G

G — 3 — I

I — 3 — J

A — 4 — B

B — 4 — D

B — 4 — C

G — 4 — J

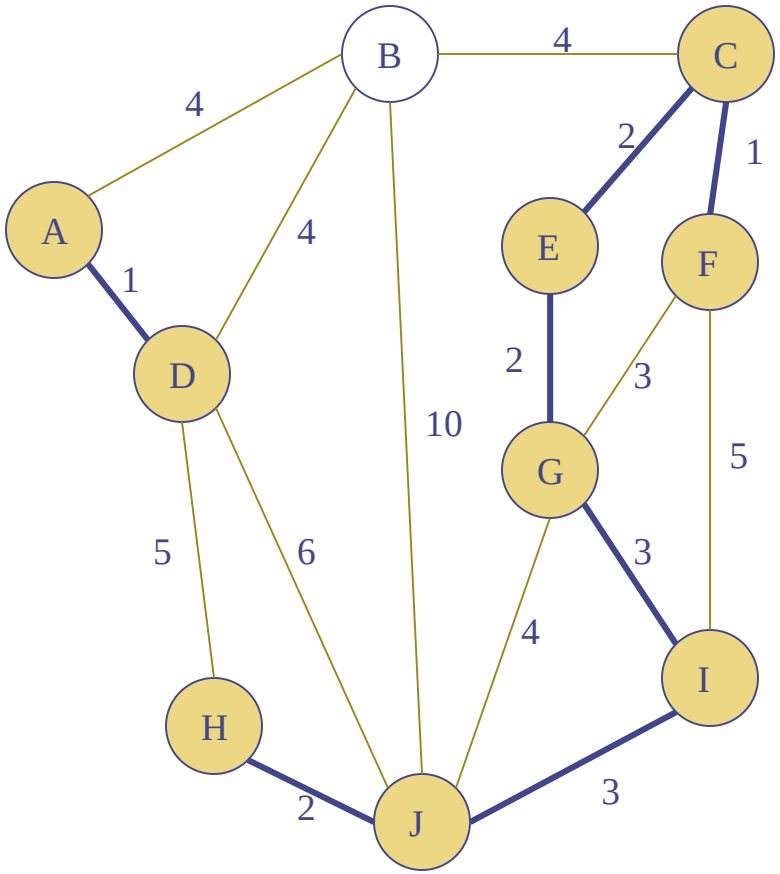
F — 5 — I

D — 5 — H

D — 6 — J

B — 10 — J

Add Edge



A — 1 — D

C — 1 — F

C — 2 — E

E — 2 — G

H — 2 — J

F — 3 — G

G — 3 — I

I — 3 — J

A — 4 — B

B — 4 — D

B — 4 — C

G — 4 — J

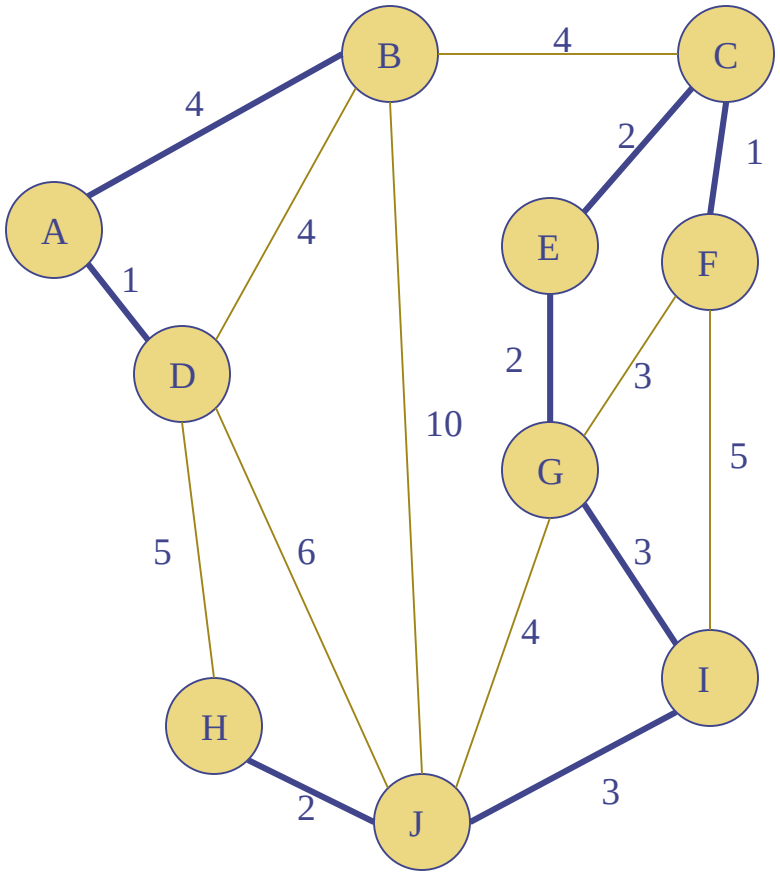
F — 5 — I

D — 5 — H

D — 6 — J

B — 10 — J

Add Edge



A — 1 — D

C — 1 — F

C — 2 — E

E — 2 — G

H — 2 — J

F — 3 — G

G — 3 — I

I — 3 — J

A — 4 — B

B — 4 — D

B — 4 — C

G — 4 — J

F — 5 — I

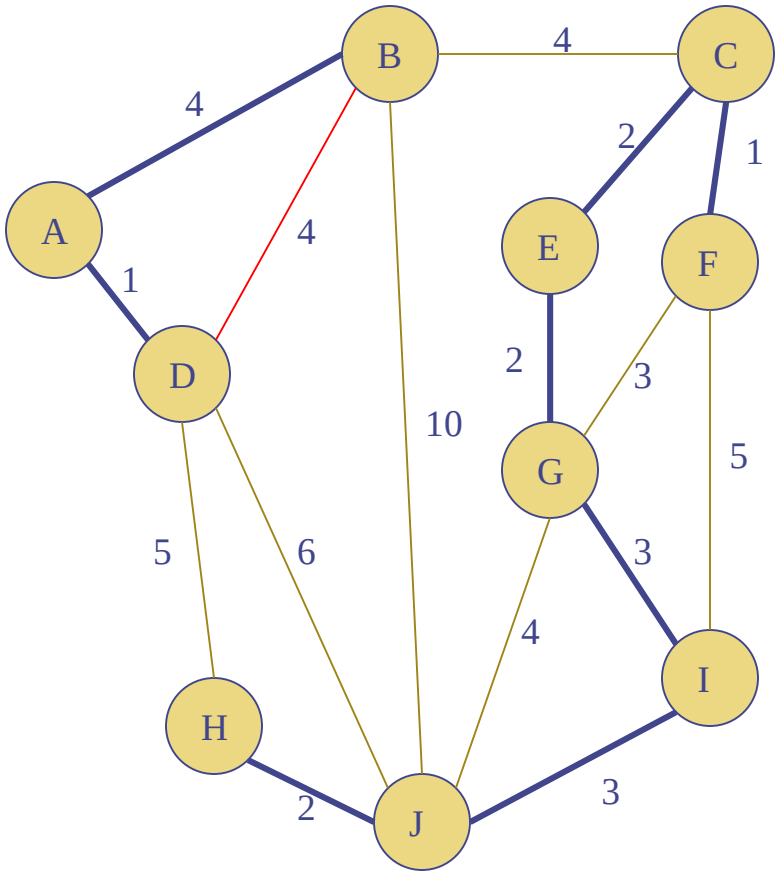
D — 5 — H

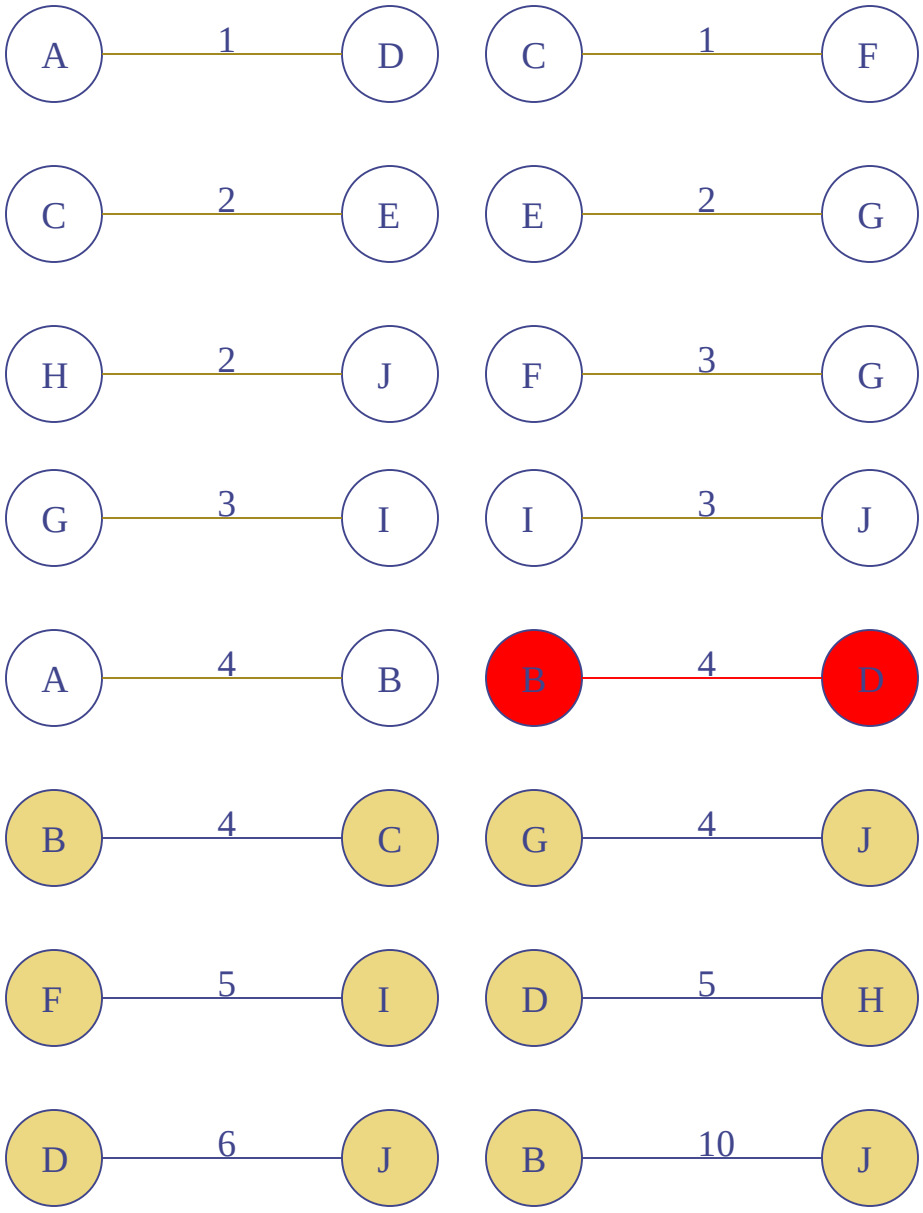
D — 6 — J

B — 10 — J

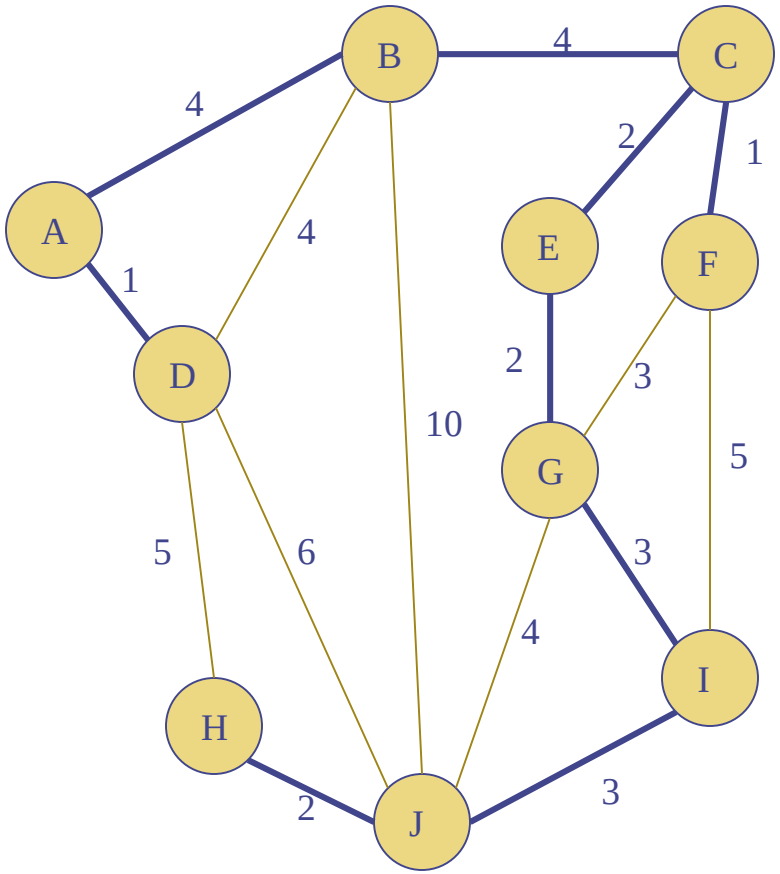
Cycle

Don’t Add Edge





Add Edge



A — 1 — D

C — 1 — F

C — 2 — E

E — 2 — G

H — 2 — J

F — 3 — G

G — 3 — I

I — 3 — J

A — 4 — B

B — 4 — D

B — 4 — C

G — 4 — J

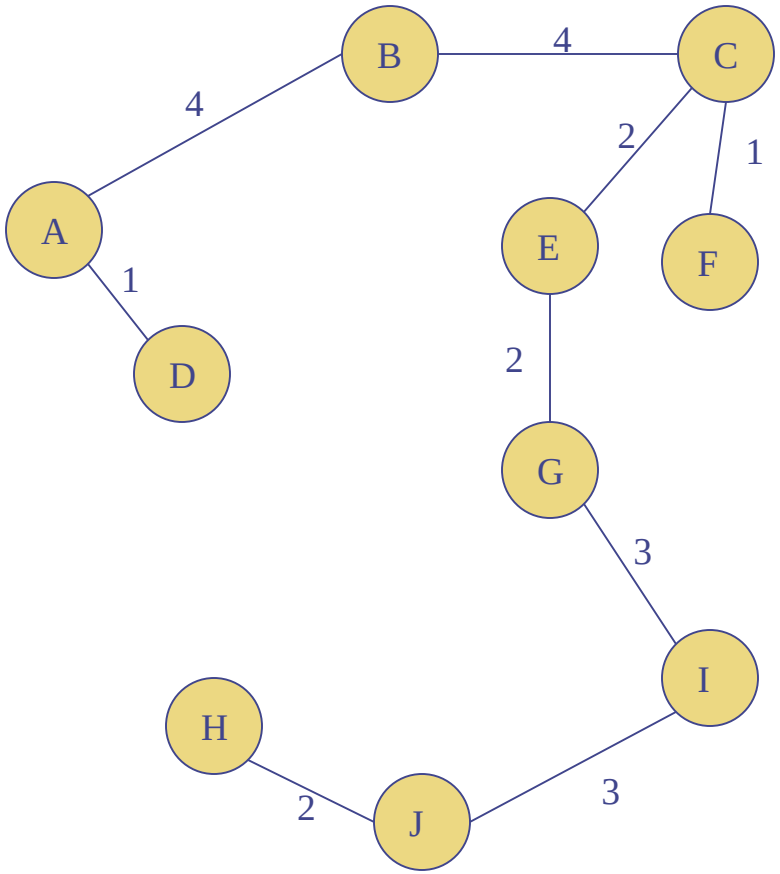
F — 5 — I

D — 5 — H

D — 6 — J

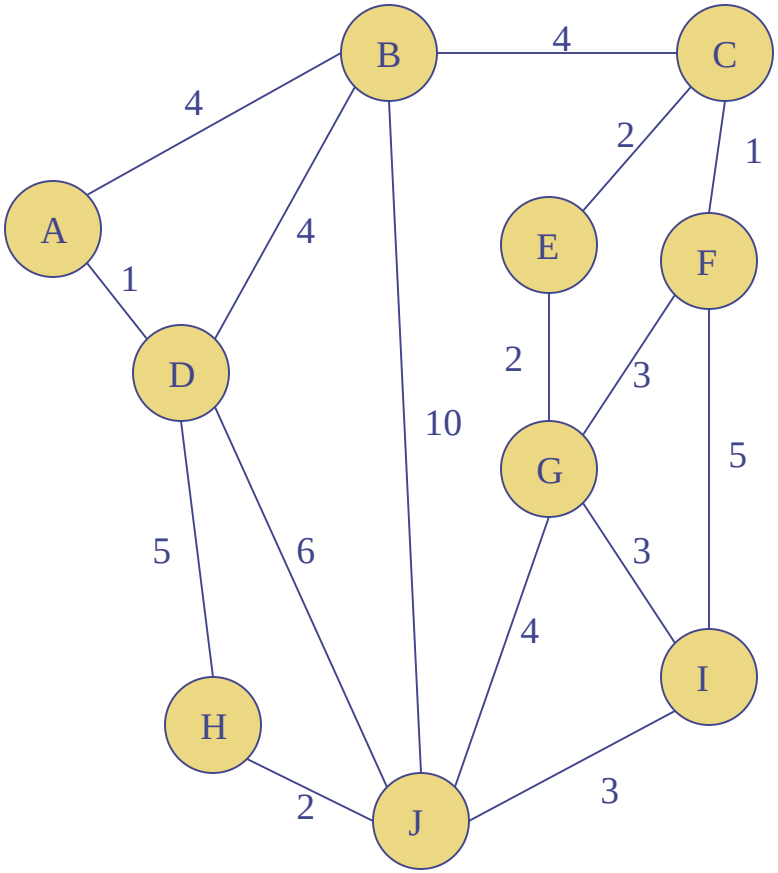
B — 10 — J

Minimum Spanning Tree



Value of MCST = 22

Complete Graph



Analysis of Kruskal's Algorithm

```
MST-KRUSKAL(G, w)
1  A = ∅
2  for each vertex v ∈ G.V
3       MAKE-SET(v)
4  sort the edges of G.E into nondecreasing order by weight w
5  for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
6       if FIND-SET(u) ≠ FIND-SET(v)
7            A = A ∪ {(u, v)}
8            UNION(u, v)
9  return A
```

Analysis of Kruskal's Algorithm

- The running time of Kruskal's Algorithm is :
  - Line 1: initialize set A is O(1)
  - Line 2-3: Make-Set() is |V|.
  - Line 4: sort the edges is O(E lg E).
  - Line 5: for loop is O(E)
  - Line 6-8: find and union by rank is O(lg V) time

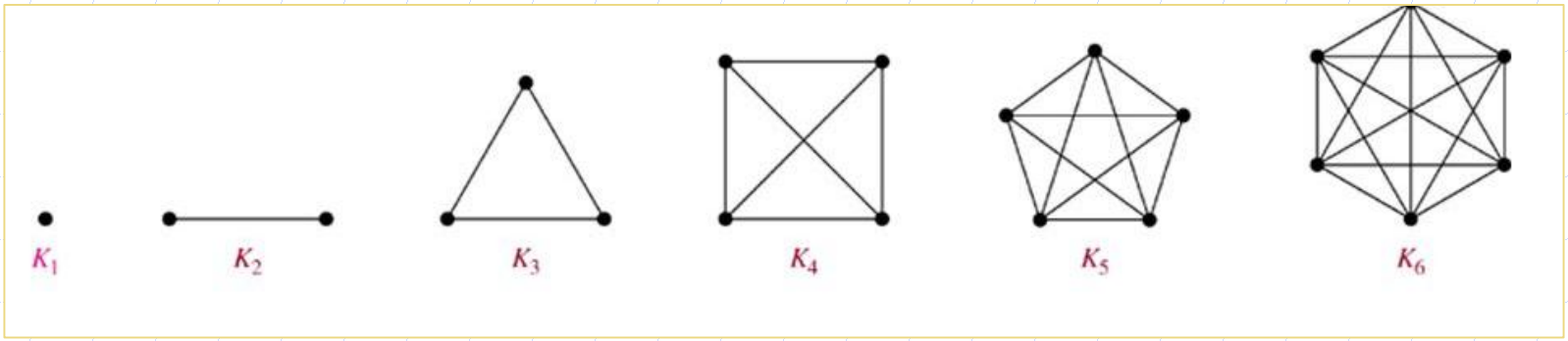
  D = {c,n,b} => D^2 = D x D = {c,n,b} x {c,n,b} = {(c,c), (c,n),(c,b), (n,c), (n,n), (n,b), (b,c), (b,n), (b,b)}
- Overall complexity is O(E lg E + E lg V) time.

  Note: lgV = lgE (Why?): E can be at most $O(V^2)$.
  - $E \rightarrow \{(x,y) | (x,y) \in V^2 \wedge x \neq y\}$ every edge maps to an ordered pair of *distinct* vertices.
  - O(E lg E) = O(E lg V^2) = O(E 2lg V) = O(E lg V) time.

So, overall complexity is O(E lg E) or O(E lg V)

Extra Slide: Analysis of Kruskal's Algorithm

- $E \rightarrow \{(x,y) \mid (x,y) \in V^2 \wedge x \neq y\}$ every edge maps to an ordered pair of *distinct* vertices.
- Example:{a, b, c}
  - V=3 , V^2 =9.
  - E= {(a,b), (a,c), (b,a), (b,c), (c,a), (c,b)} – {(a,a),(b,b),(c,c)}



Prim's Algorithm

- In Prim’s algorithm, the set A forms a single tree. The safe edge added to A is always a least-weight edge connecting the tree to a vertex not in the tree.
- The tree starts from an arbitrary root vertex r and grows until the tree spans all the vertices in V .
- Each step adds to the tree A a light edge that connects A to an isolated vertex—one on which no edge of A is incident.
- Prim’s algorithm operates much like Dijkstra’s algorithm for finding shortest paths in a graph.

Prim's Algorithm

Steps of Prim's Algorithm :

- Pick any vertex v.
- Choose the *shortest edge* from v to any other vertex w outside the tree.
- Add the edge (v,w) to the MCST.
- Continue to add at every step the *shortest edge* from a vertex in the MCST (New Graph), to a vertex outside (Old Graph), with out worrying about overall structure.
- Stop when all the vertices are in the MCST

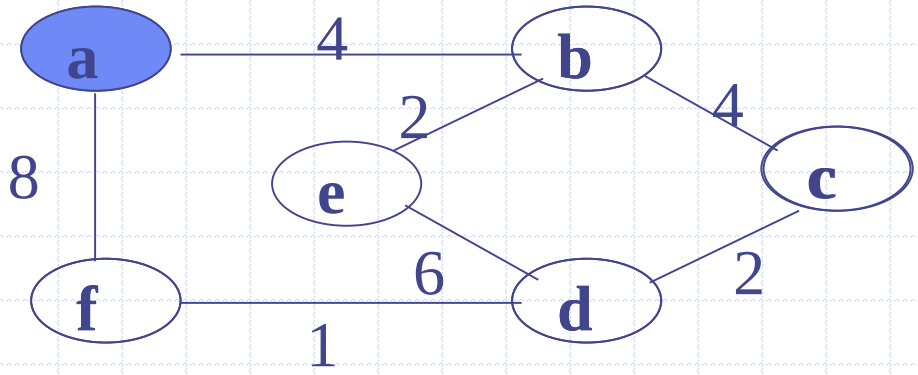
Prim's Algorithm

```
MST-PRIM(G, w, r)
 1  for each u ∈ G.V
 2      u.key = ∞
 3      u.π = NIL
 4  r.key = 0
 5  Q = G.V
 6  while Q ≠ ∅
 7      u = EXTRACT-MIN(Q)
 8      for each v ∈ G.Adj[u]
 9          if v ∈ Q and w(u, v) < v.key
10              v.π = u
11              v.key = w(u, v)
```

- all vertices that are not in the tree reside in a queue Q based on a key attribute.
- The attribute v.Key is the minimum weight of any edge connecting v to a vertex in the tree

Prim's Algorithm: example 1

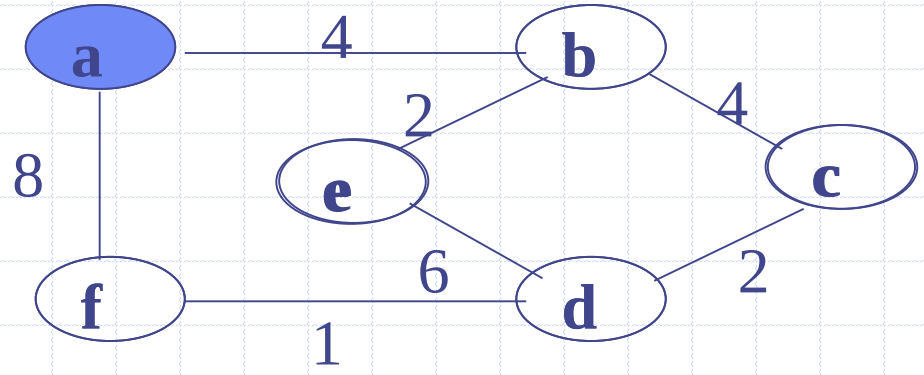
Initially



Nodes	a	b	c	d	e	f
Key	0	∞	∞	∞	∞	∞
Π	Nil	Nil	Nil	Nil	Nil	Nil
Q	a	b	c	d	e	f

Prim's Algorithm: example 1

Initially



Nodes	a	b	c	d	e	f
Key	0	∞	∞	∞	∞	∞
Π	Nil	Nil	Nil	Nil	Nil	Nil
Q	a	b	c	d	e	f

Step 1

Extract_Min(Q) => a

Q	b	c	d	e	f

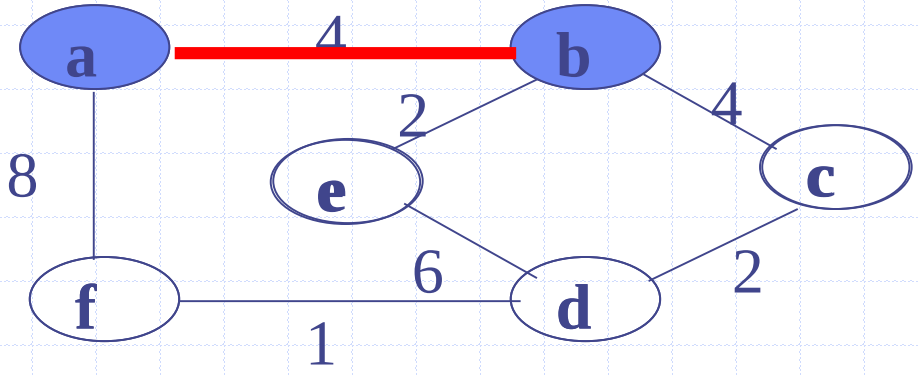
Adj[a]	b	f

b Є Q and Key[b] > w(a, b) => π[b] = a, Key[b] = 4

f Є Q and Key[f] > w(a, f) => π[f] = a, Key[f] = 8

Prim's Algorithm: example 1

After Step 1



Nodes	a	b	c	d	e	f
Key	0	4	∞	∞	∞	8
Π	Nil	a	Nil	Nil	Nil	a
Q		b	c	d	e	f

Step 2

Extract_Min(Q) => b

Q	c	d	e	f

Adj[b]	a	c	e

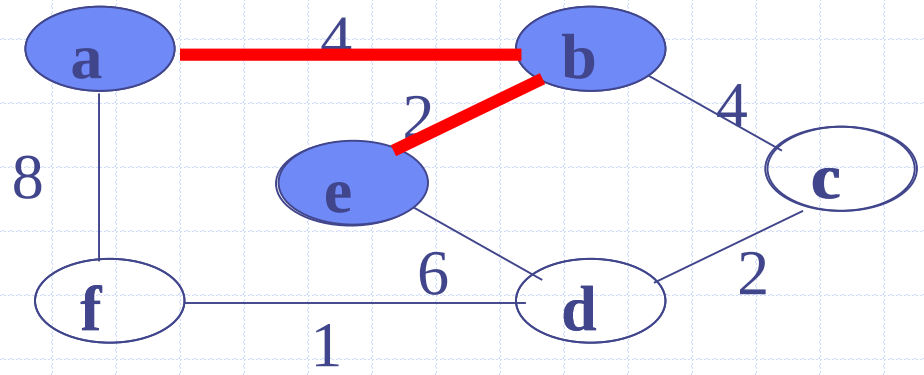
a not ϵ Q => **ignore**

c Є Q and Key[c] > w(b,c) => π[c] = b, Key[c] = 4

e Є Q and Key[e] > w(b,e) => π[e] = b, Key[e] = 2

Prim's Algorithm: example 1

After Step 2



Nodes	a	b	c	d	e	f
Key	0	4	4	∞	2	8
Π	Nil	a	b	Nil	b	a
Q			c	d	e	f

Step 3

Extract_Min(Q) => e

Q	c	d	f

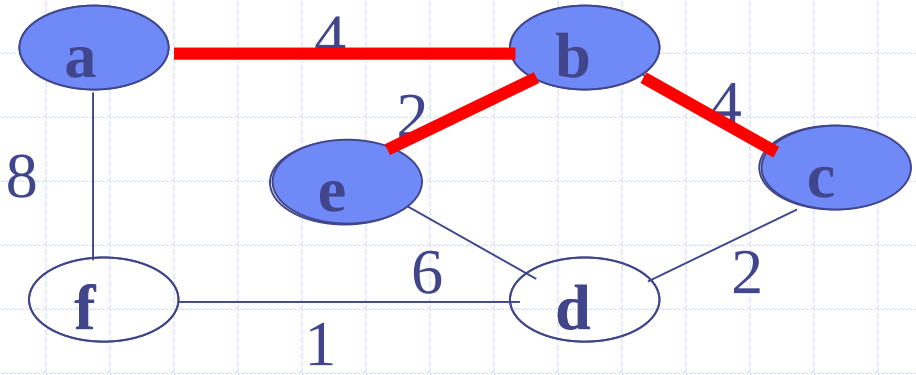
Adj[e]	b	d

b not ϵ Q => **ignore**

d Ε Q and Key[d] > w(e, d) => Π[d] = e, Key[d] = 6

Prim's Algorithm: example 1

After Step 3



Nodes	a	b	c	d	e	f
Key	0	4	4	6	2	8
Π	Nil	a	b	e	b	a
Q			c	d		f

Step 4

Extract_Min(Q) => c

Q	d	f

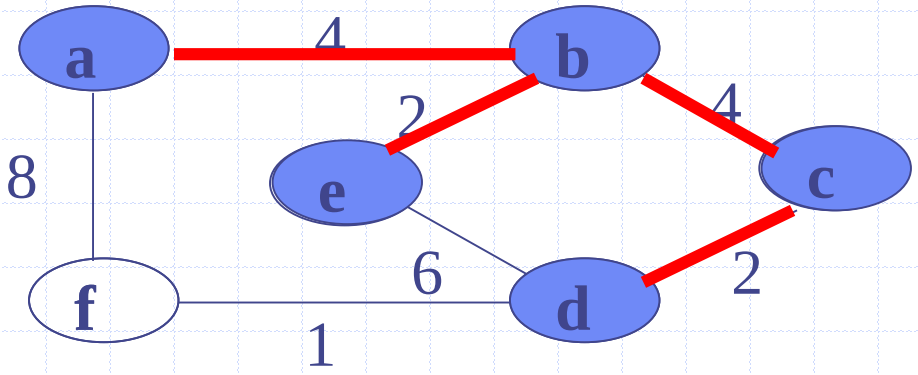
Adj[c]	b	d

b not ϵ Q => **ignore**

d Ε Q and Key[d] > w(c, d) => π[d] = c, Key[d] = 2

Prim's Algorithm: example 1

After Step 4



Nodes	a	b	c	d	e	f
Key	0	4	4	2	2	8
Π	Nil	a	b	c	b	a
Q				d		f

Step 5

Extract_Min(Q) => d

Q	f

Adj[d]	c	e	f

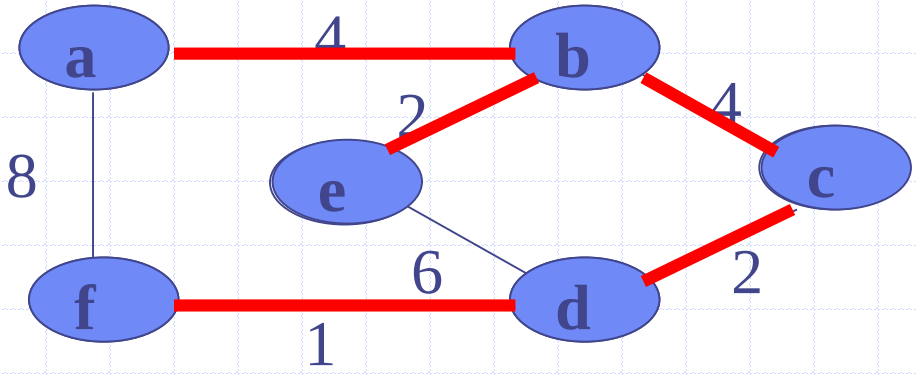
c not ϵ Q => **ignore**

e not ϵ Q => **ignore**

f Є Q and Key[f] > w(d, f) => π[f] = d, Key[d] = 1

Prim's Algorithm: example 1

After Step 5



Nodes	a	b	c	d	e	f
Key	0	4	4	2	2	1
Π	Nil	a	b	c	b	d
Q						f

Step 6

Extract_Min(Q) => f

Q	---

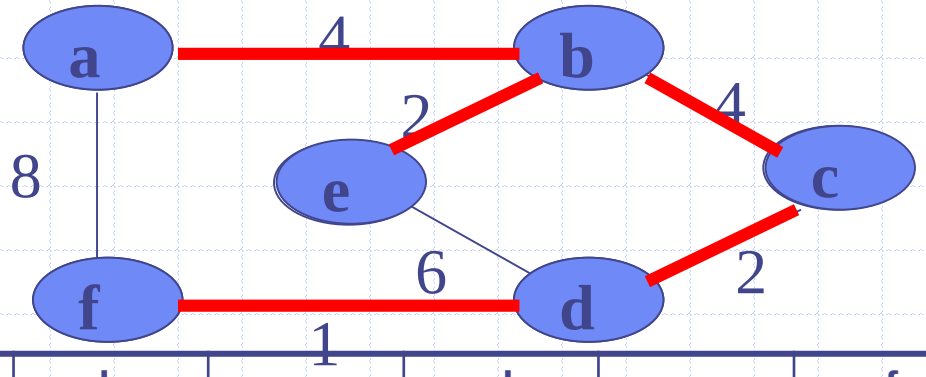
Adj[f]	a	d

a not ϵ Q => **ignore**

d not ϵ Q => **ignore**

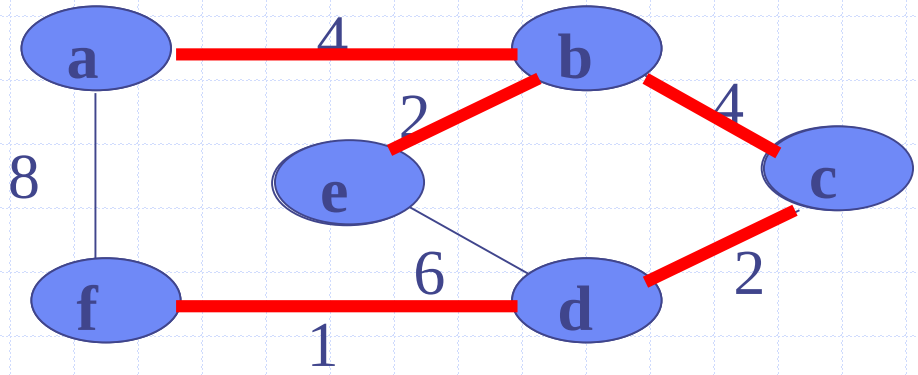
Prim's Algorithm: example 1

After Step 6



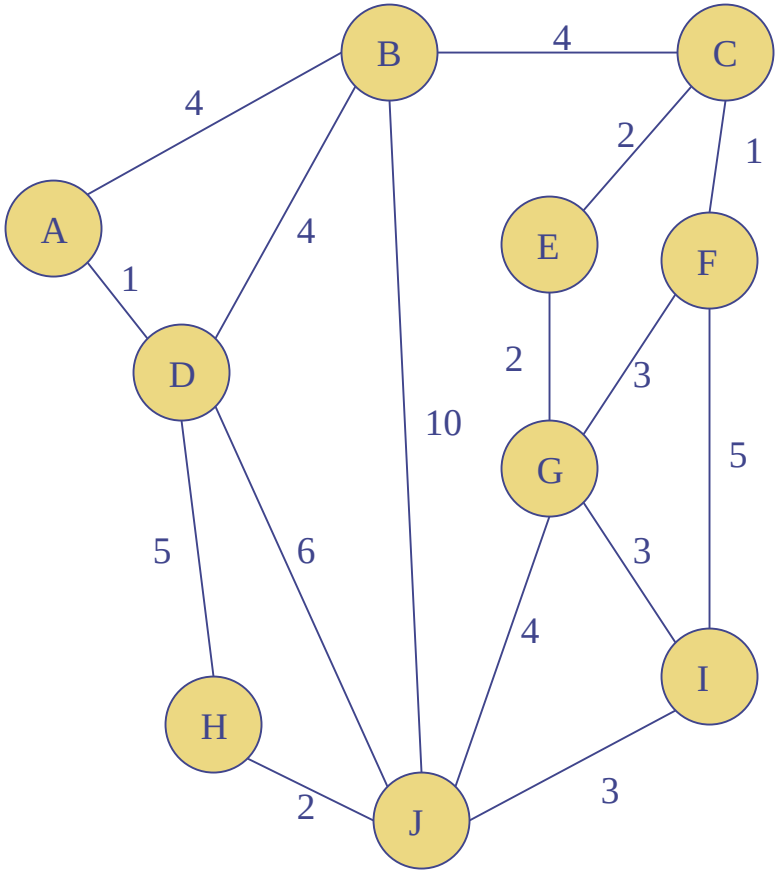
Nodes	a	b	c	d	e	f
Key	0	4	4	2	2	1
Π	Nil	a	b	c	b	d
Q						

MCST

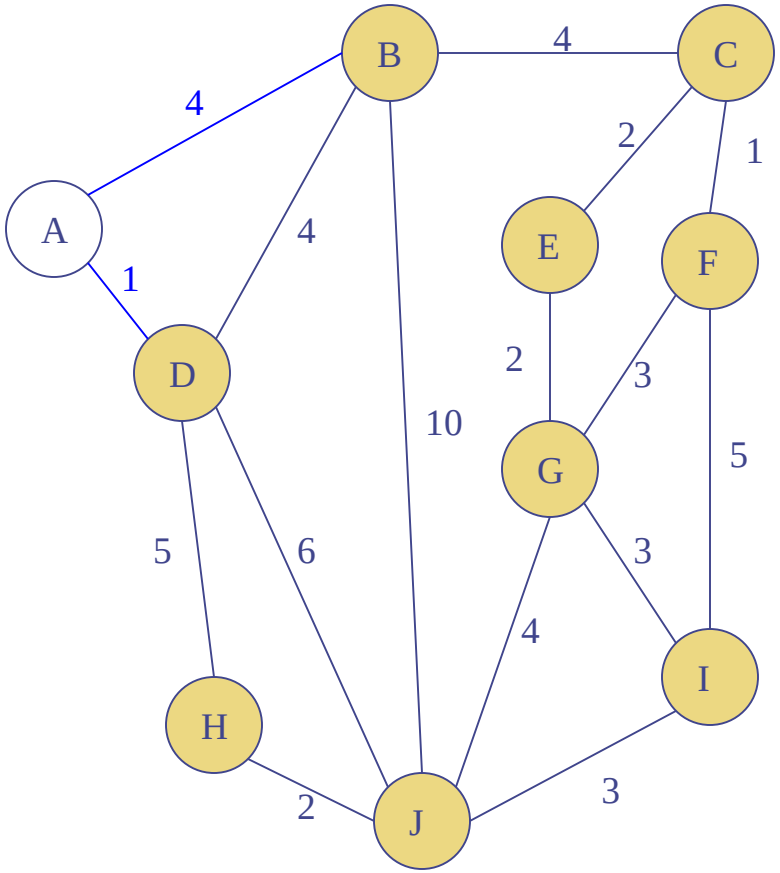


Value of MCST = 4 + 4 + 2 + 2 + 1 = 13.

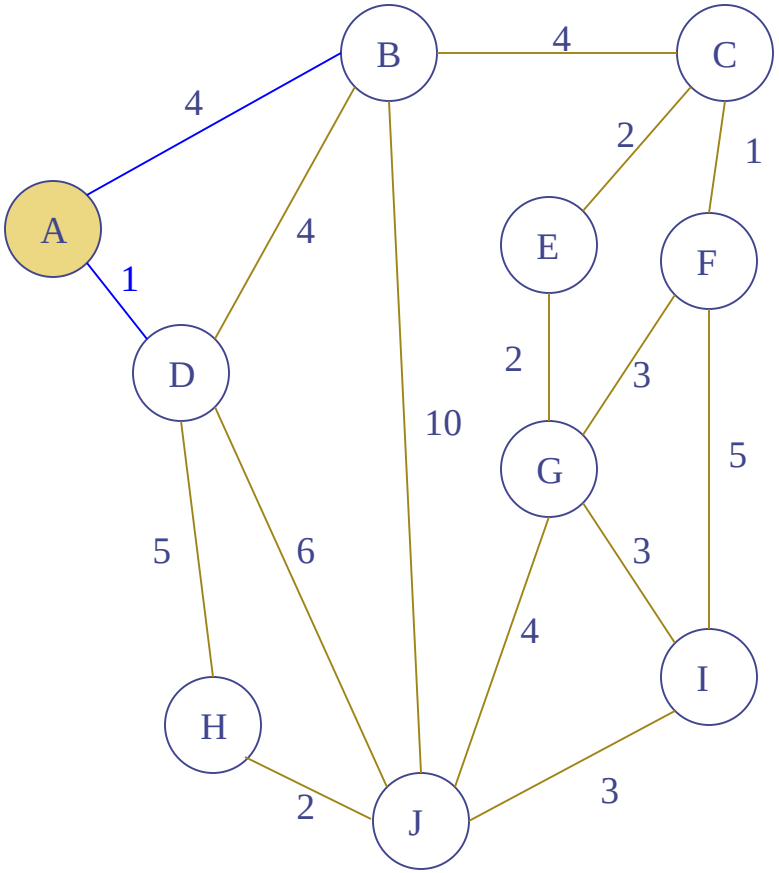
Prim's Algorithm: example 2



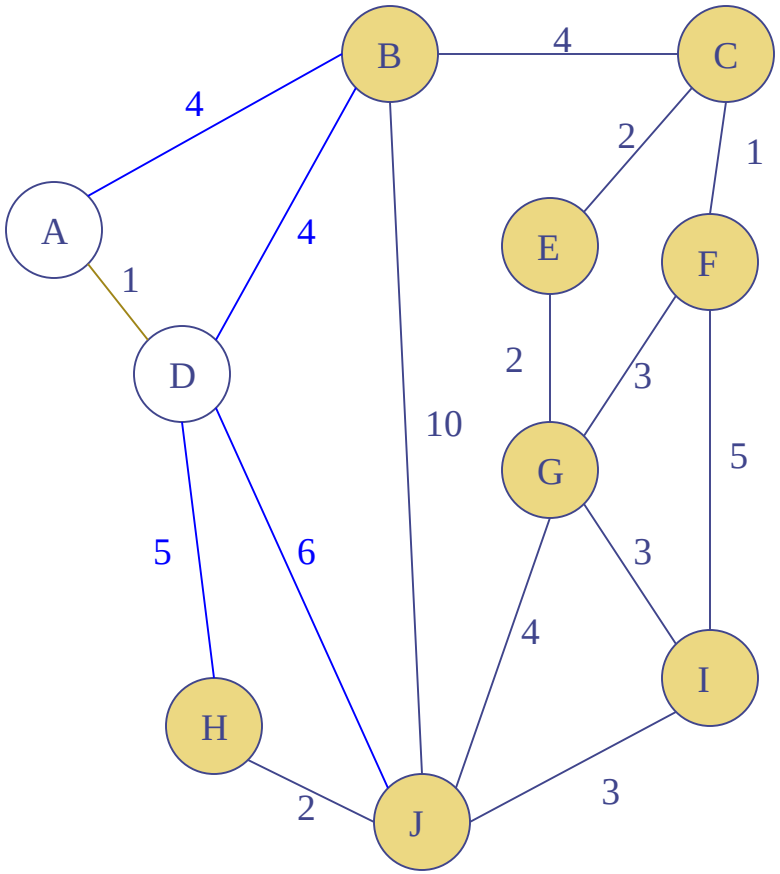
Old Graph



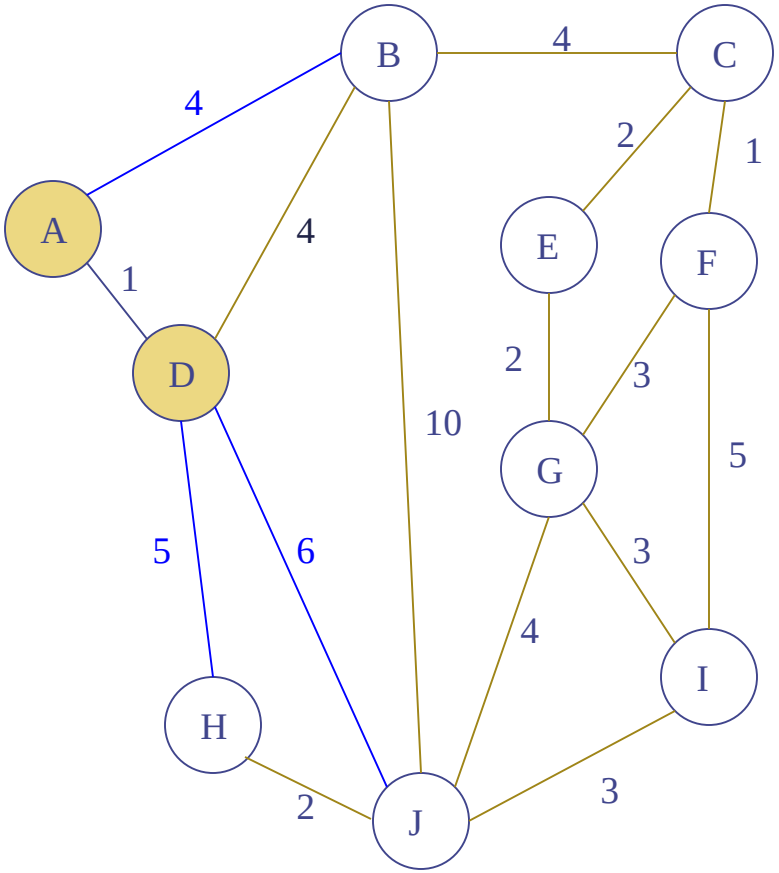
New Graph



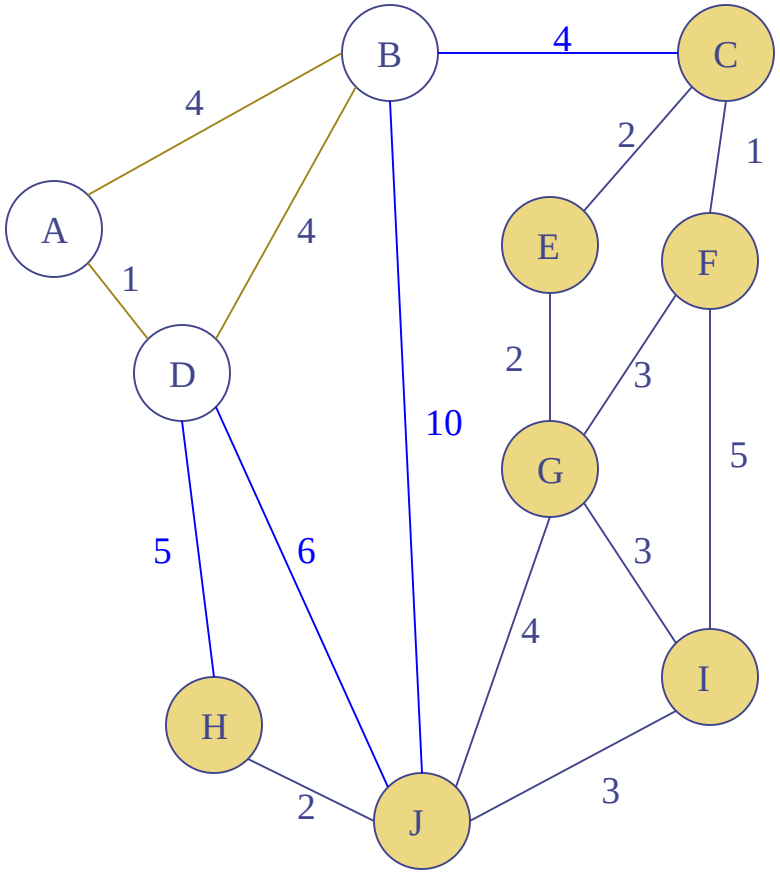
Old Graph



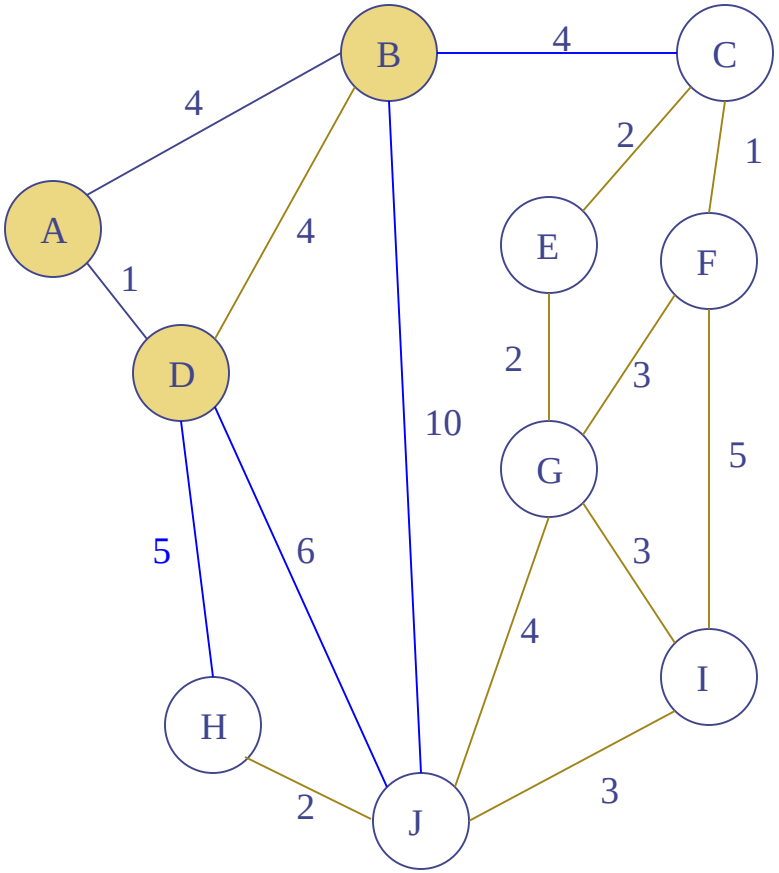
New Graph



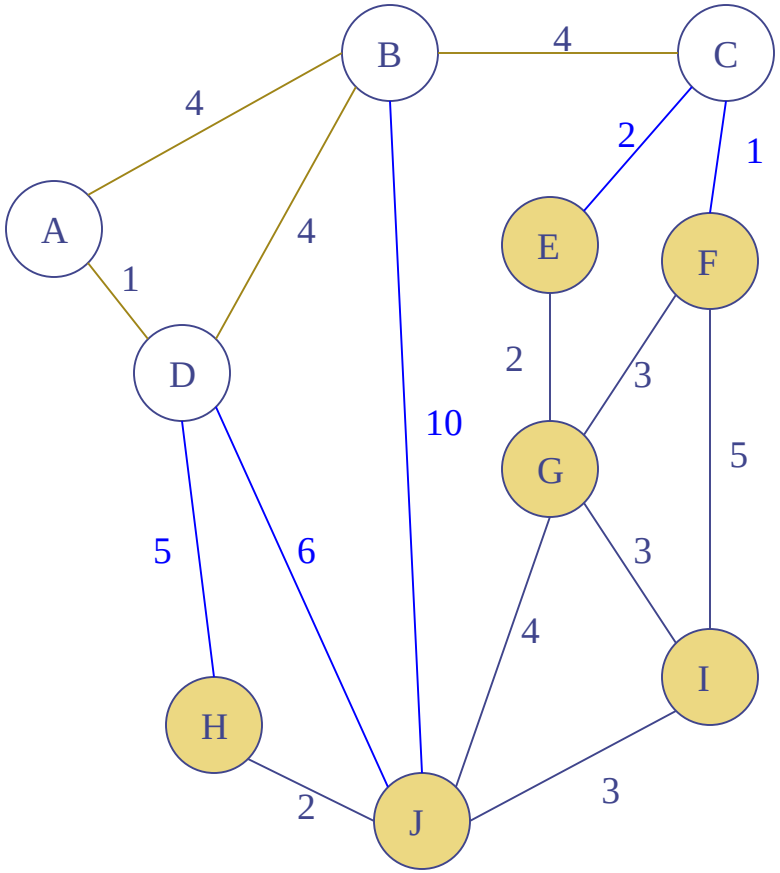
Old Graph



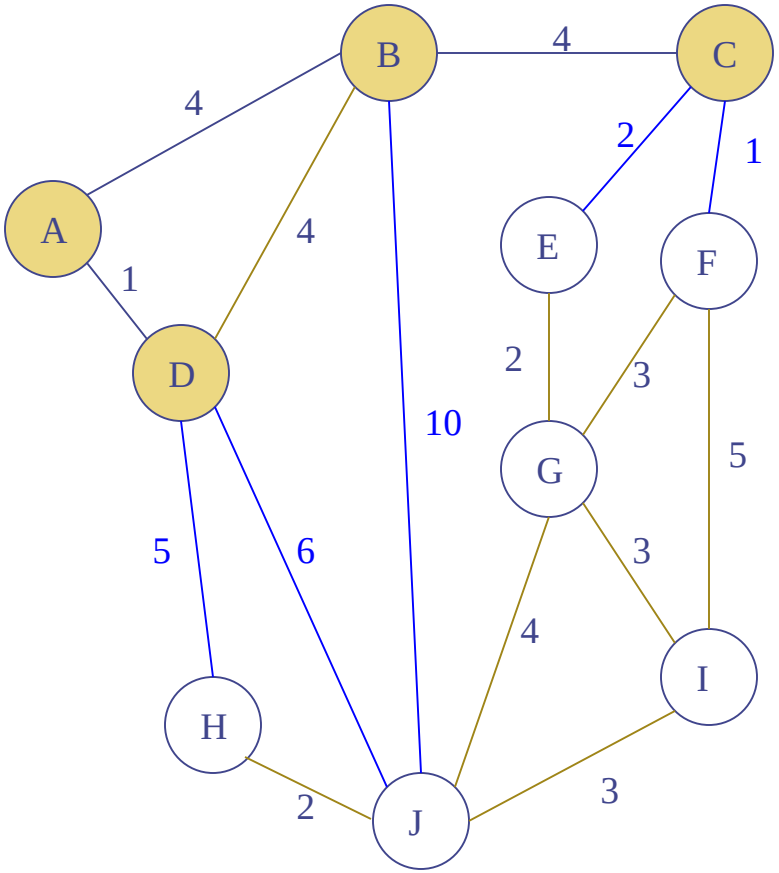
New Graph



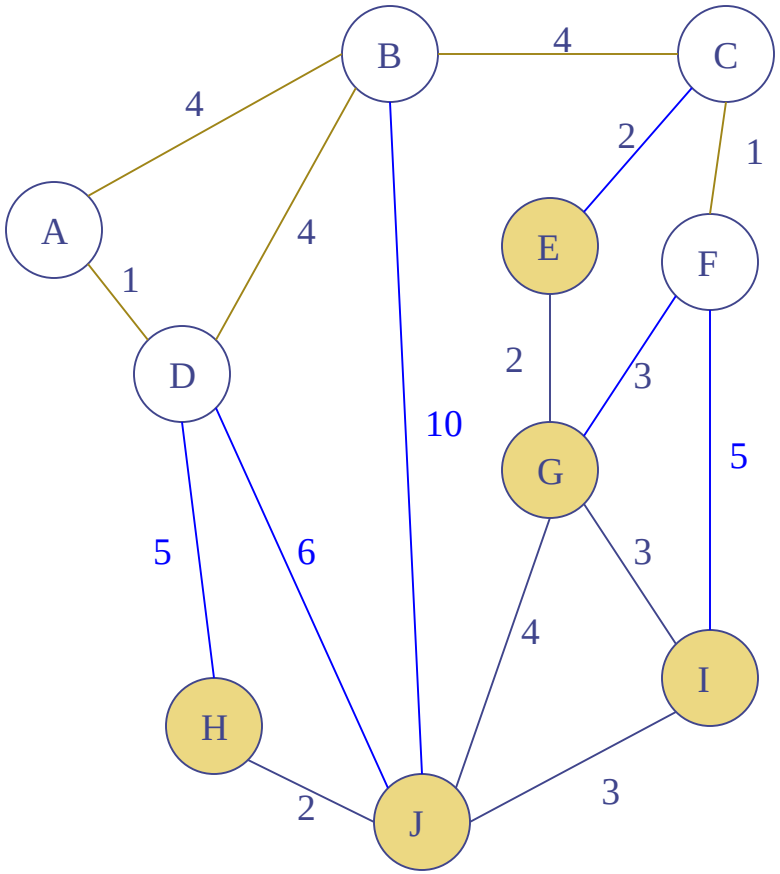
Old Graph



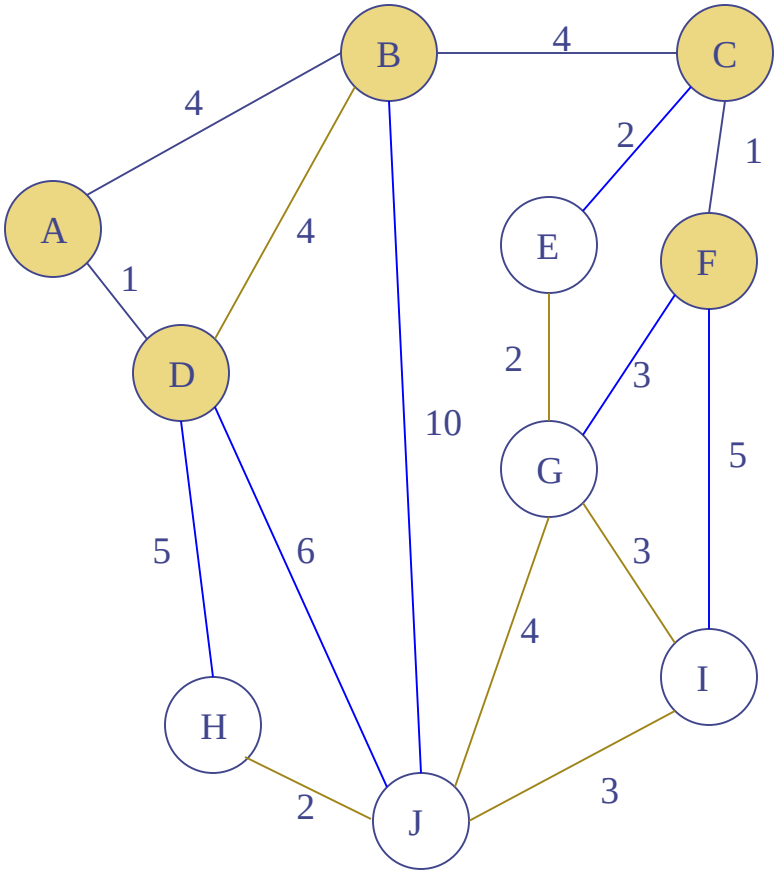
New Graph



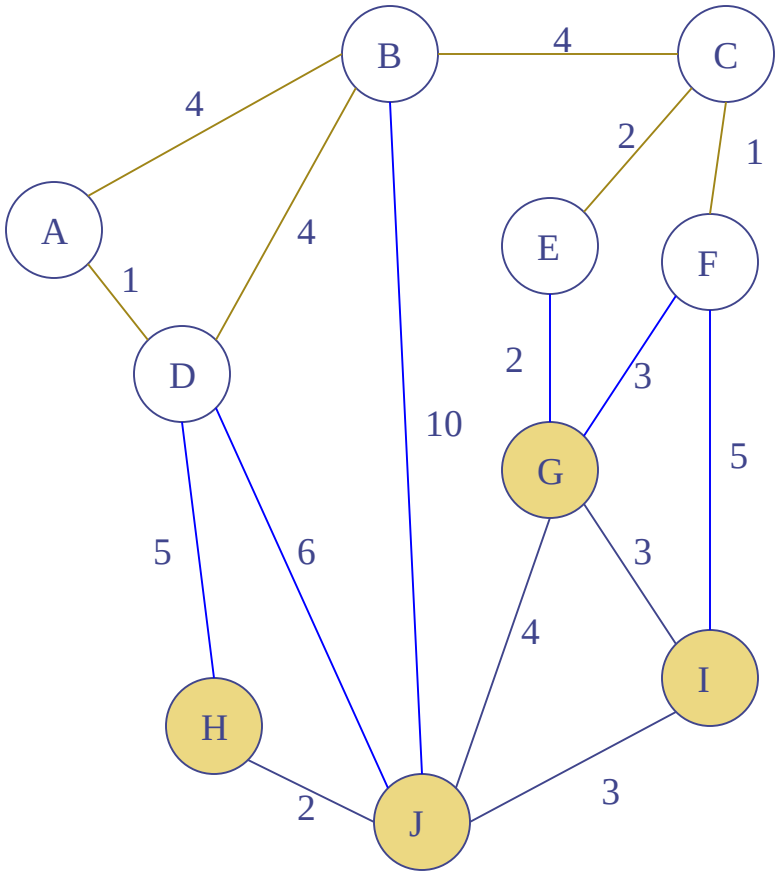
Old Graph



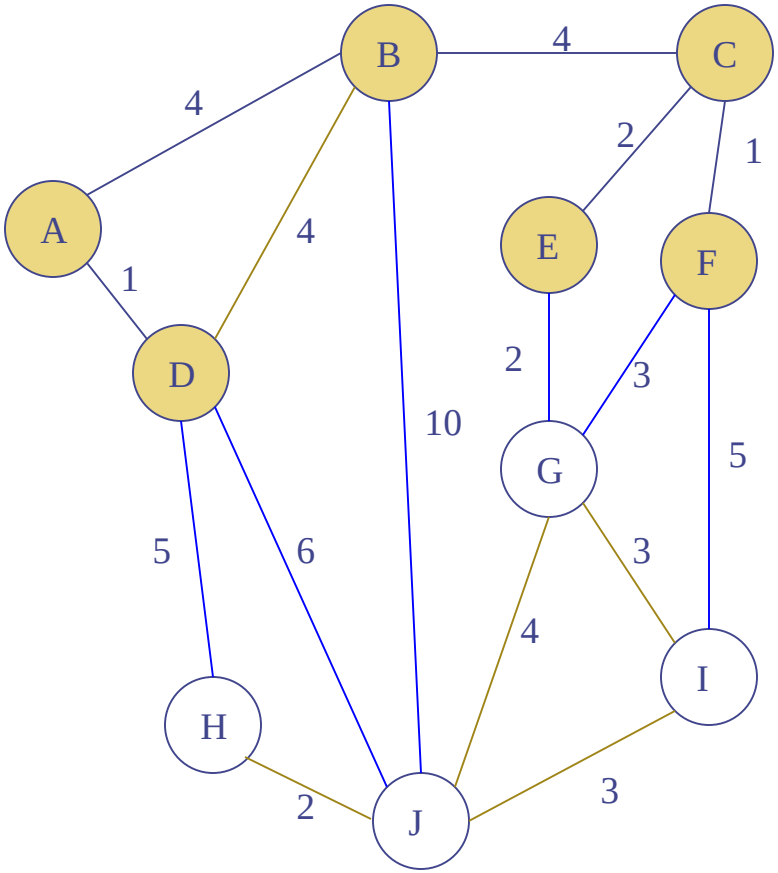
New Graph



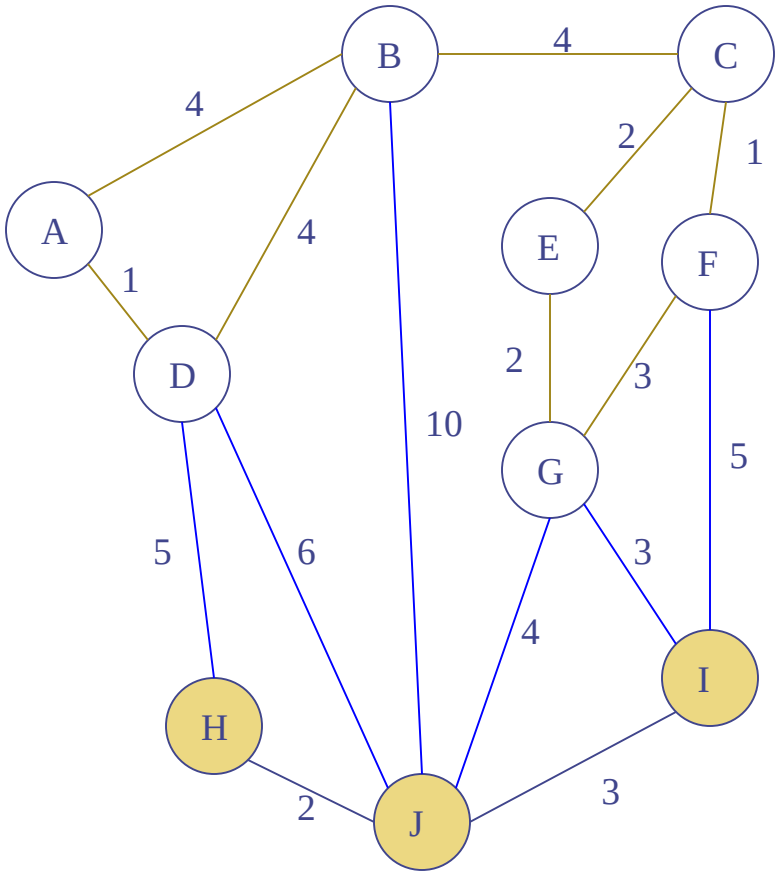
Old Graph



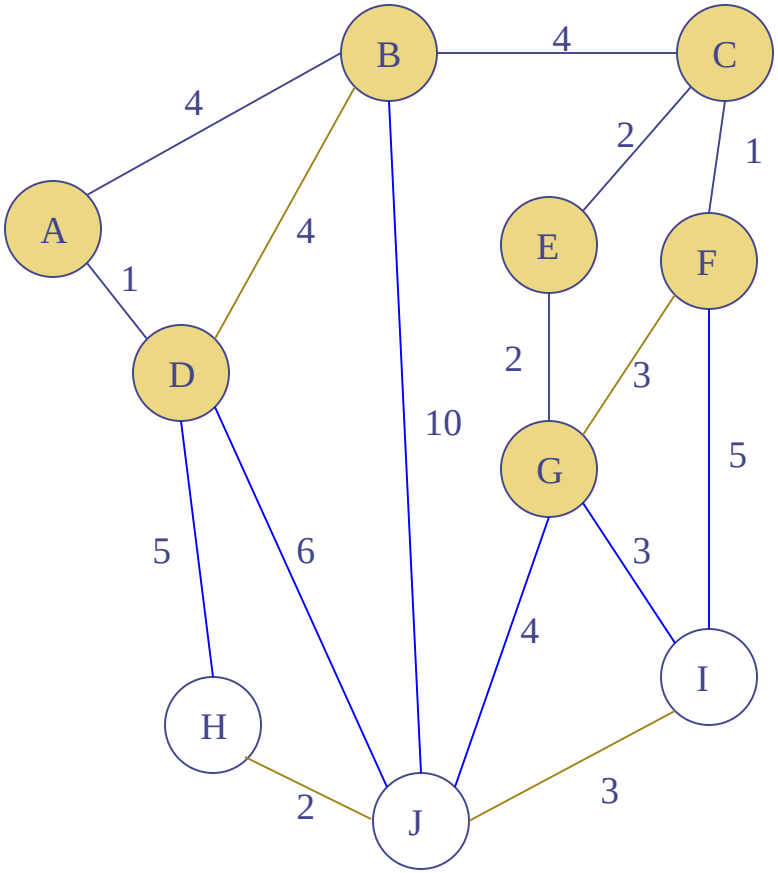
New Graph



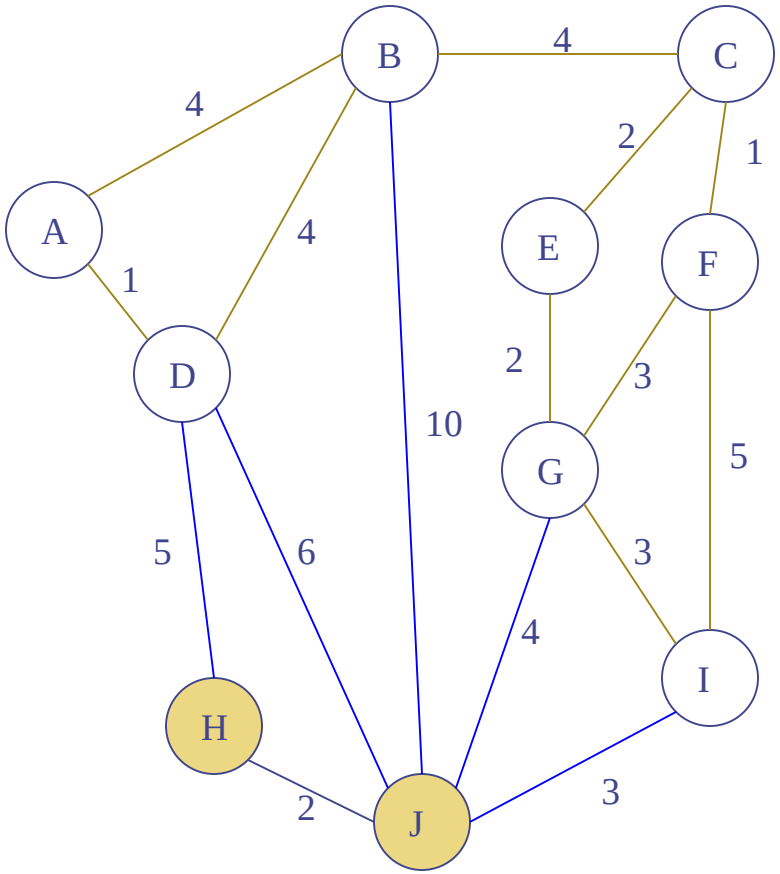
Old Graph



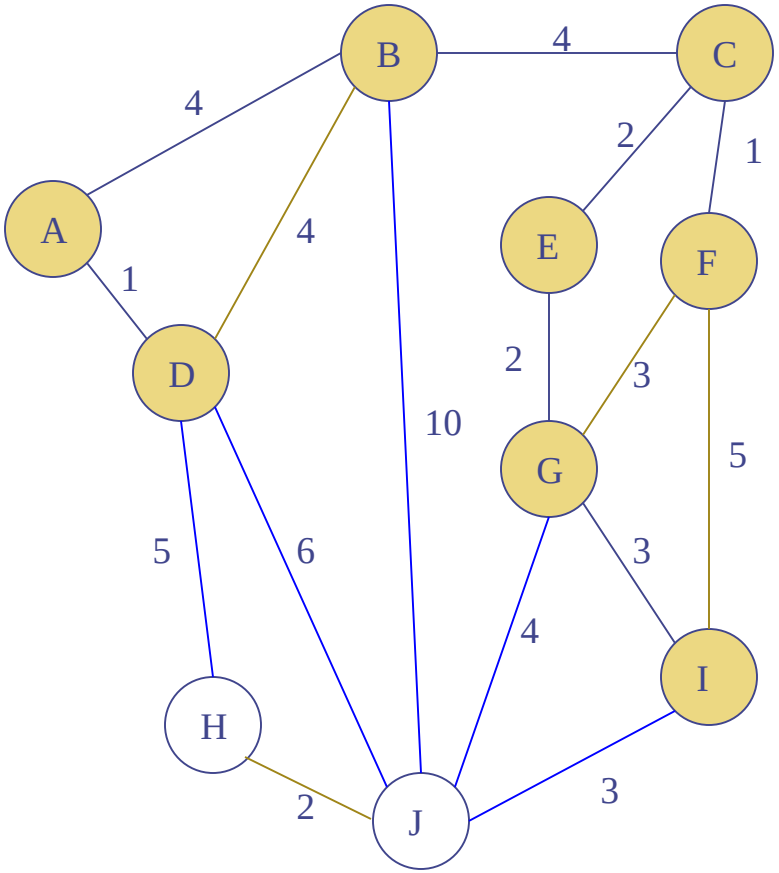
New Graph



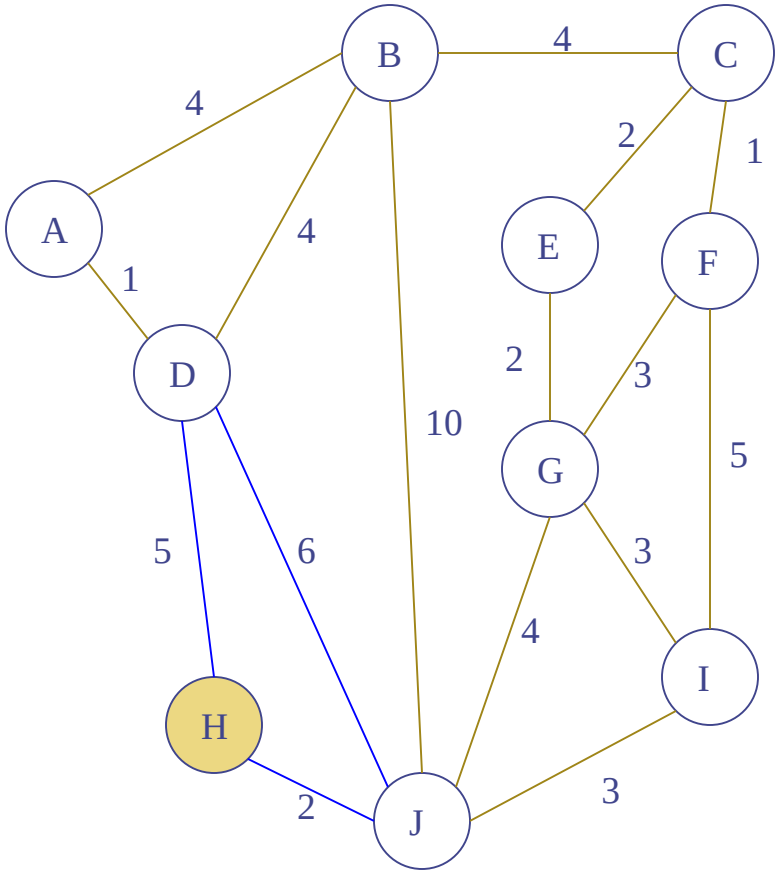
Old Graph



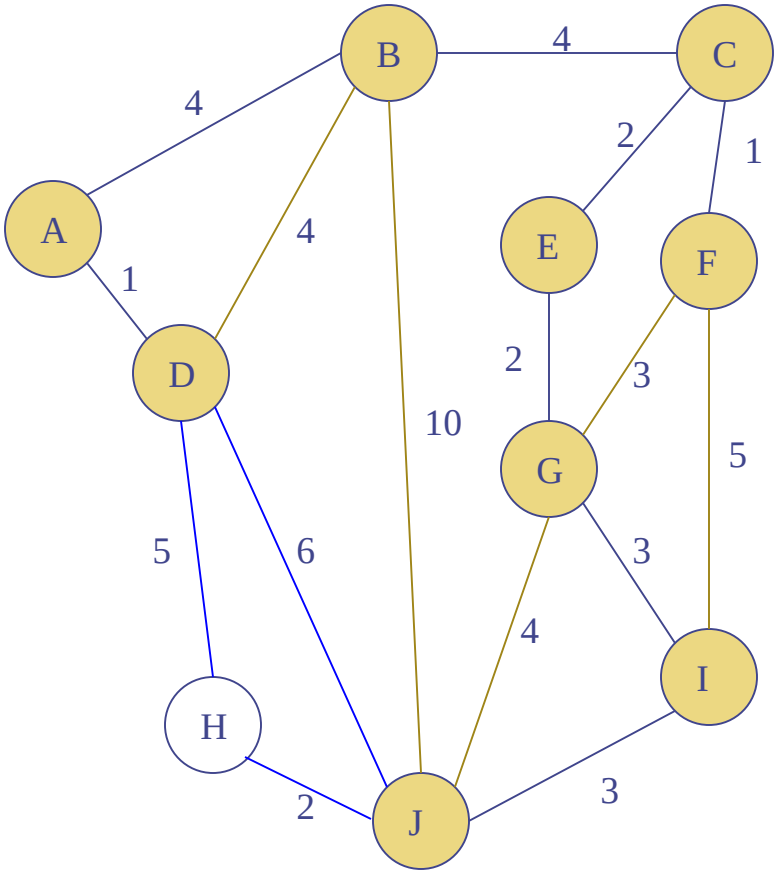
New Graph



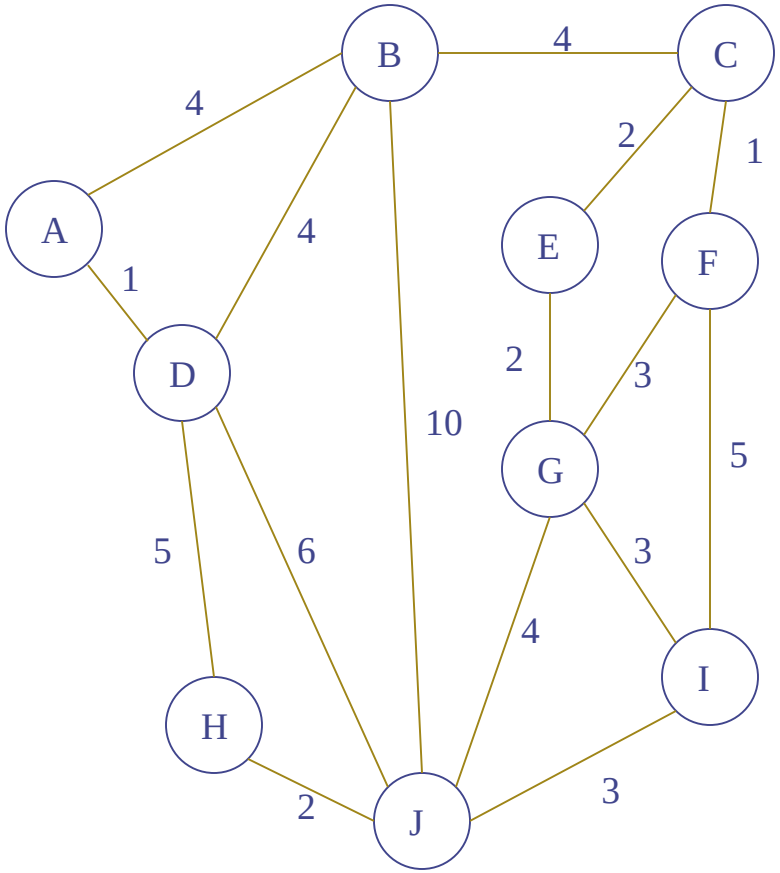
Old Graph



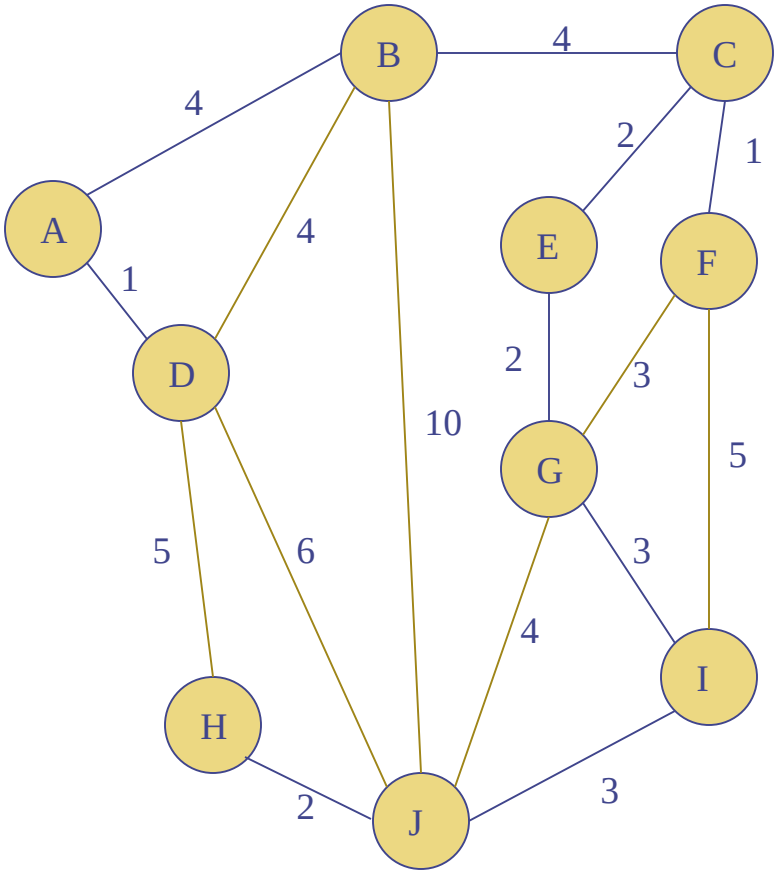
New Graph



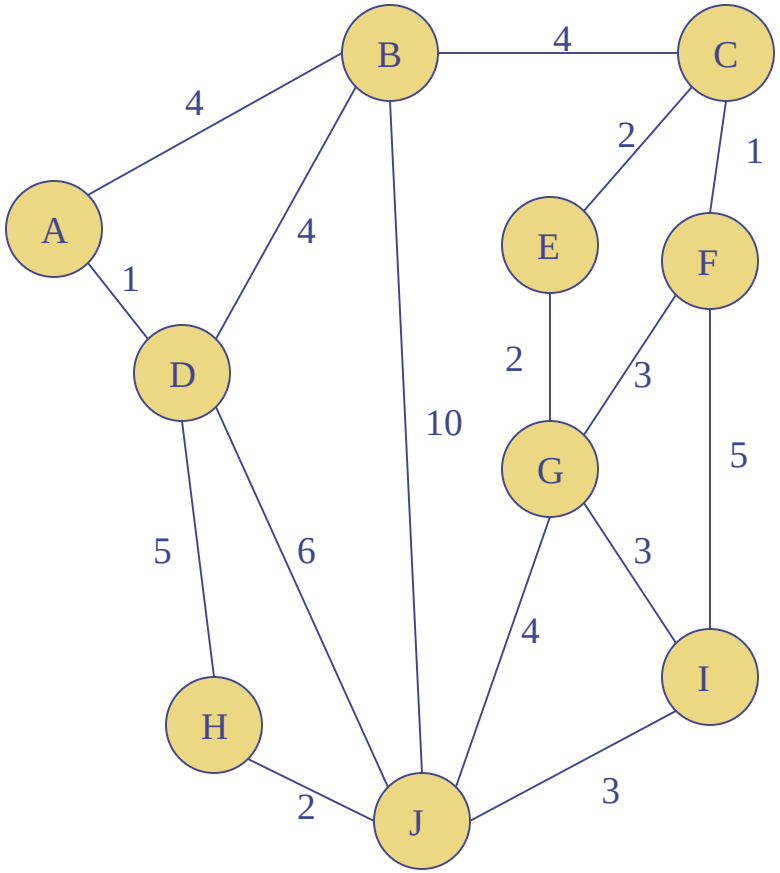
Old Graph



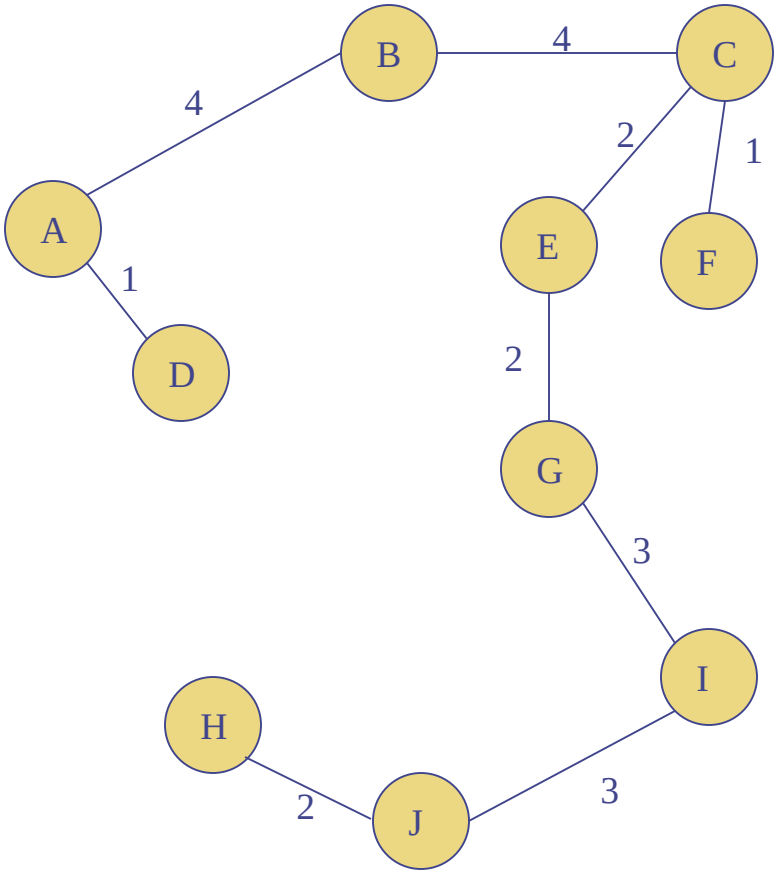
New Graph



Complete Graph



Minimum Spanning Tree



Value of MCST = 22

Analysis of Prim's Algorithm

- The running time of Prim's algorithm depends on how we implement the min-priority queue Q.
  - will build it as a binary min-heap.
- The running time of Prim's algorithm is asymptotically same as for Kruskal's algorithm.

Analysis of Prim's Algorithm

```
MST-PRIM(G, w, r)
 1  for each u ∈ G.V
 2      u.key = ∞
 3      u.π = NIL
 4  r.key = 0
 5  Q = G.V
 6  while Q ≠ ∅
 7      u = EXTRACT-MIN(Q)
 8      for each v ∈ G.Adj[u]
 9          if v ∈ Q and w(u, v) < v.key
10              v.π = u
11              v.key = w(u, v)
```

Analysis of Prim's Algorithm

- Line 1-5: Initializing the key, parent and Queue take **O(V)** time.
  - By using the BUILD-MIN-HEAP procedure.
- Line 6: The body of **while** loop is executed **|V|** times.
- Line 7: Each Extract-Min() operation takes **O(lg V)** time.
  - To call MIN-HEAPIFY() after deleting the root (min value).
- Line 7: So, total time for all calls to Extract-Min() is **O(V lg V)**.
- Line 8: The **for** loop is executed **O(E)** times.
- Line 9-10: In constant time **O(1)**.
- Line 11: Decreasing key value on the min-heap is **O(lg V)** time.
- The total running time of the algorithm is **O(V lg V+ E lg V)=O(E lg V).**